

# Your AI Agent Did Something in Production. Can You Explain Exactly What Happened?

A production architecture for tracing, auditing, governing and proving autonomous agent behaviour, with a recorded POC that breaks it on purpose and measures what logs, traces and execution lineage can each reconstruct.

T5

## T5 · Production AI Engineering

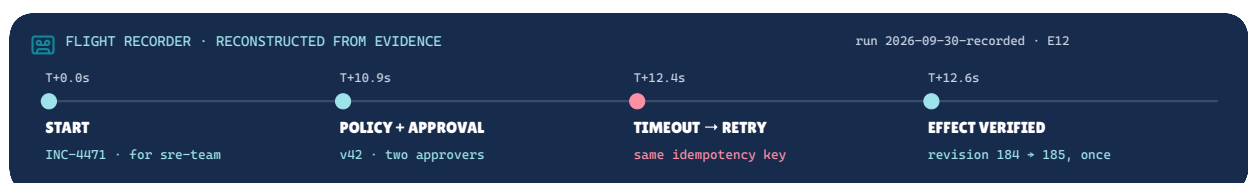
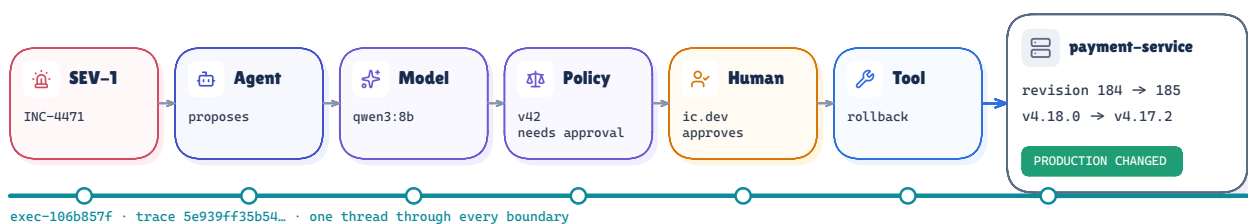
53 min read · Published 2026-09-30 · Technical deep dive

— PRODUCTION AI ENGINEERING · T5 · OBSERVABILITY & GOVERNANCE

## Your agent changed production. Can you prove why?

6 hours later...  
security asks why.

One execution, from intent to physical side effect,  
recorded so it can be explained after the runtime is gone.



The reference edition. Every measured number is substituted from the recorded POC run; every external claim links to a numbered, verified source.

## About this edition

The Medium edition tells one story: a production rollback that worked, and the six hours later when nobody could say exactly what had happened. This edition is the reference behind it, for platform engineers, SREs, architects and security engineers who have to answer that question for real agents. It covers:

- what an agent execution has to record for its production changes to be explainable later, and why ordinary logs and traces leave the important parts out;
- a correlation model (execution, action, attempt, external transaction) that survives processes, networks, restarts and retries;
- why *the tool returned 200* is not evidence that production changed, and how to record what did change;
- how operational telemetry and governance evidence differ in schema, retention, access, durability and integrity, although they start from the same events;
- what a recorded POC measured when it ran fifteen scenarios through three observation layers at once and scored each against ground truth.

Three kinds of statement appear, and they are marked:

- **Sourced.** An external fact with a numbered reference, for example the OpenTelemetry GenAI conventions <sup>[9]</sup>. Every source was fetched and quoted in `research/sources.md`, including what it does *not* support.
- **Our synthesis.** An architectural position this series takes, such as "the lineage key, not the log store, is the centre of the design".
- **Implemented / measured.** Behaviour of the T5 POC. Numbers are substituted from the recorded runs' `facts.json` at build time; none is typed by hand.

## 1. The rollback worked. Now prove what happened.

The incident is the one this series has followed since F3. At 14:02 a Datadog monitor fires: payment-service is failing 14.2% of requests against a 1% SLO, thirteen minutes after release v4.18.0 shipped. The monitor invokes `incident-agent-prod` under the delegation `group:sre-team` holds (T1). The agent reads the runbook and the deployment history, and a model proposes a rollback to v4.17.2. The policy engine evaluates the proposal against `prod-rollback` and returns `ALLOW_WITH_APPROVAL` (T2). Two eligible people approve that exact action (T3). The tool gateway calls the deployment API. The deployment moves from revision 184 to 185, the error rate falls, and the incident resolves.

Everything worked.

Six hours later, the security team and the incident review ask the questions that always come after an autonomous change to production:

- Who initiated the rollback, and on whose authority?
- Which agent, which version, and which model and configuration produced the proposal?
- Which policy evaluated it, which version of that policy, and what did it evaluate?
- Was approval required? Who could approve? Who did, and what exactly did they approve?
- Which capability executed, with which arguments, under which identity, against which resource?
- Did the side effect actually happen? Was there a retry? Could it have happened twice?
- What was the final outcome, and can all of this be shown to be unaltered?

The platform team says what every platform team says: *we have logs*.

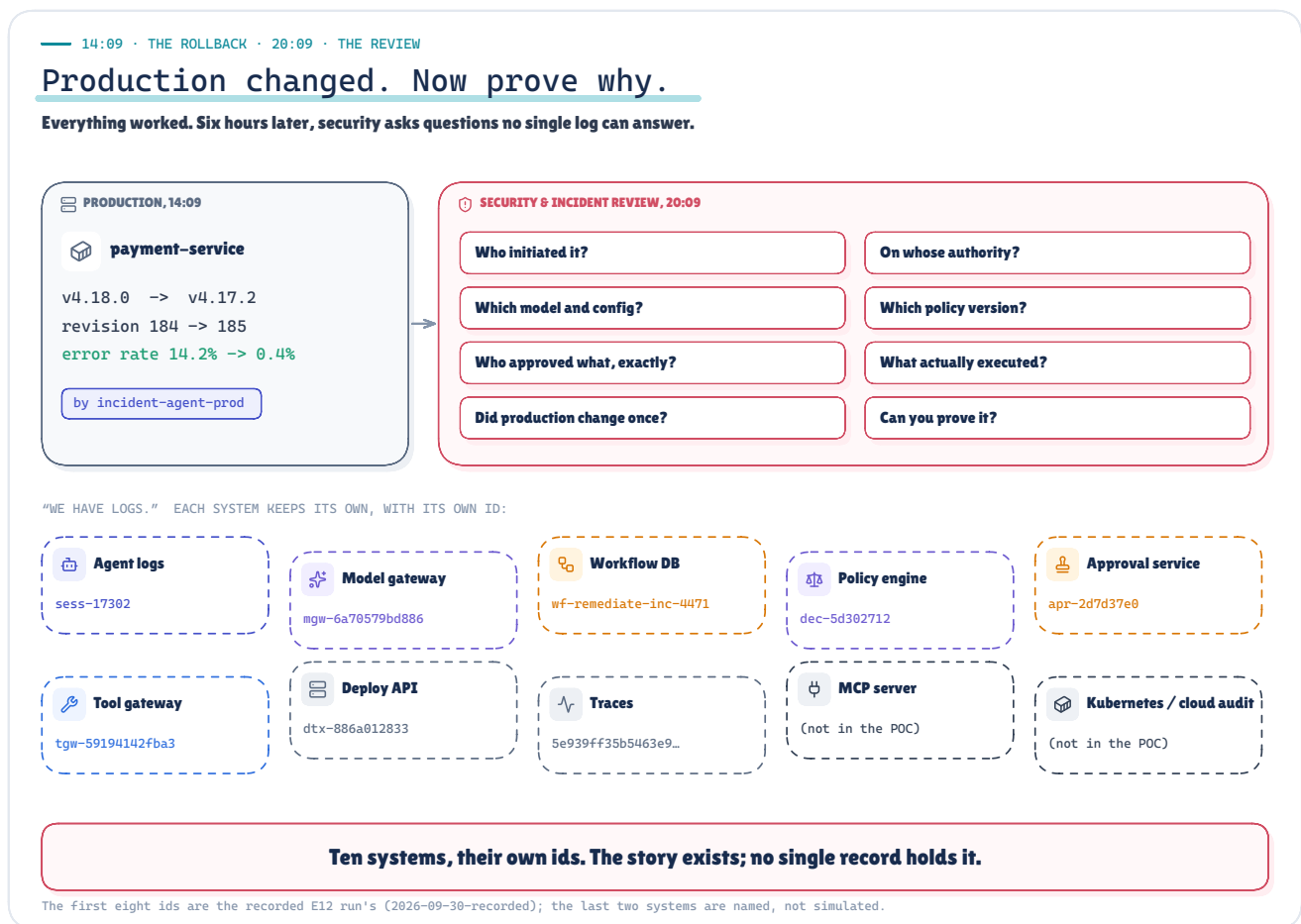


Figure 1. Everything worked. Six hours later the questions need a single story, and the ids are spread over ten systems.

Concept + recorded: the ids are E12's, from each component's own log · run 2026-09-30-recorded

The logs exist. They are spread across the agent runtime, the model gateway, the workflow database, the policy engine's decision log, the approval service, the tool gateway, the MCP server, the Kubernetes API audit log, application telemetry and the cloud provider's audit trail. Each one is accurate about its own component. Each uses its own identifiers. None of them holds the story.

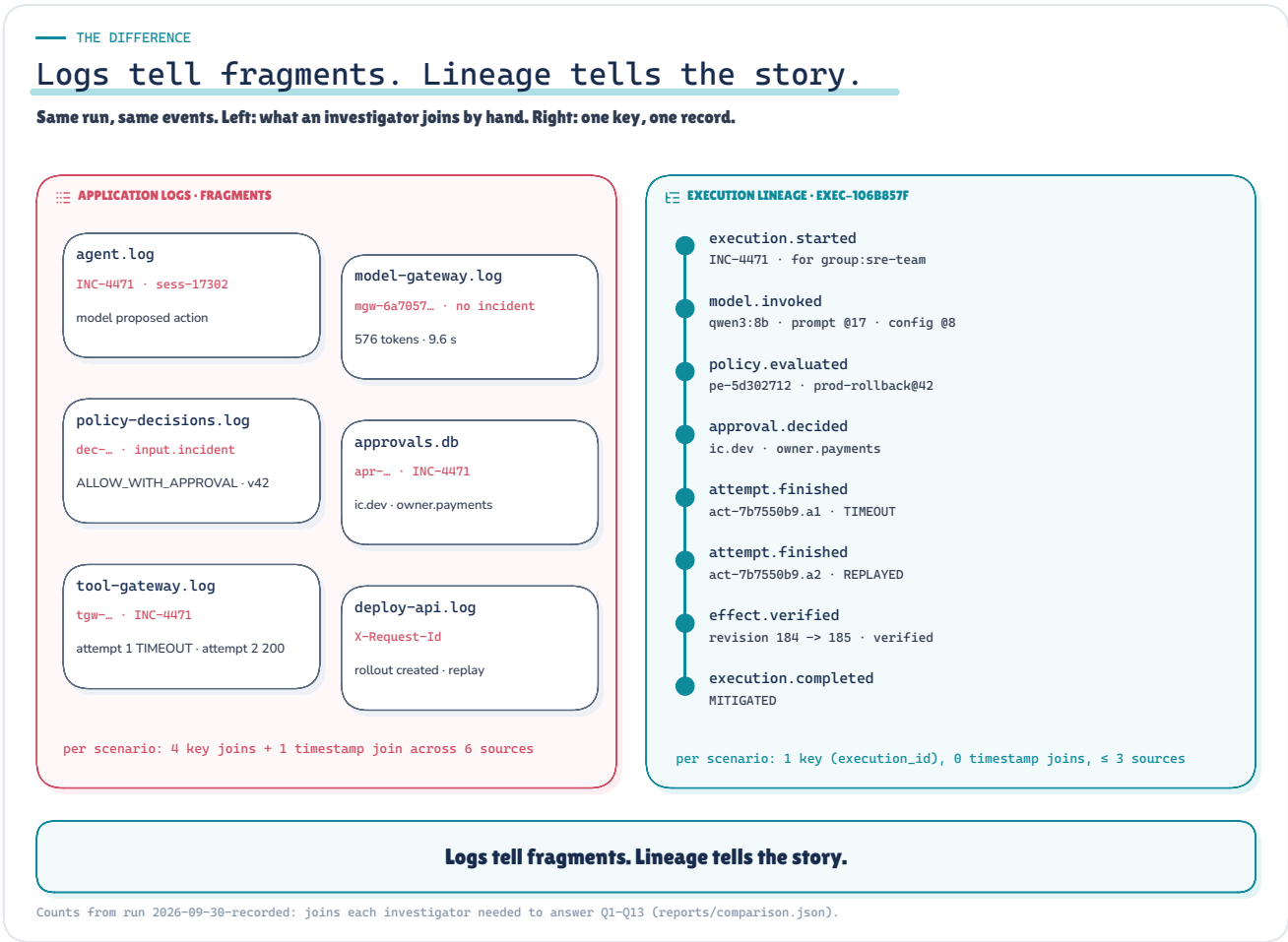
## 2. We already have logs. Why isn't that enough?

This is not an argument against logs. The POC below gave application logs a fair chance, and they did well: across fifteen recorded scenarios they answered 149 of 195 reconstruction questions correctly (76%). A decent policy decision log records its bundle revision, so the logs knew which policy version decided in every scenario (15 of 15). The tool gateway propagated `X-Request-Id`, so the logs could count attempts exactly (15 of 15).

What the logs could not do is the interesting part. They never said on whose behalf the agent acted (0 of 15). They never said which prompt template or agent configuration produced the proposal (0 of 15). And an ordinary, mutable application log could never show, by itself, that its history had not

been altered (0 of 15); some logging platforms add retention locks or WORM storage, which is exactly the kind of property that turns a log into evidence. And in the one scenario where the deployment API said "rolled back" but never applied the change, the logs recorded the incident as mitigated while production was still failing.

Adding distributed traces changed less than I expected. Logs plus OpenTelemetry traces answered exactly the same questions: 149 of 195. What traces changed was *how* the answers were found. The investigator using only logs needed 15 joins on timestamps across the run; with trace context it needed 0. That matters for investigators, because a timestamp join is a guess that happens to be right until two things happen at once. But it doesn't produce new governance facts. A trace tells you which spans belong together. It does not, by itself, know who delegated authority, which policy version was in force, or whether the deployment really changed.



The gap is not "more logging". Autonomous agents add a causal chain that ordinary application instrumentation was never designed to capture, and the parts of it that matter most to governance are exactly the parts that sit *between* components.

### 3. Autonomous agents add a new causal chain

A request/response service has a simple causal story: a caller asked, the service did something, it answered. An agent execution has a longer one, and each link is a different kind of fact owned by a different component:

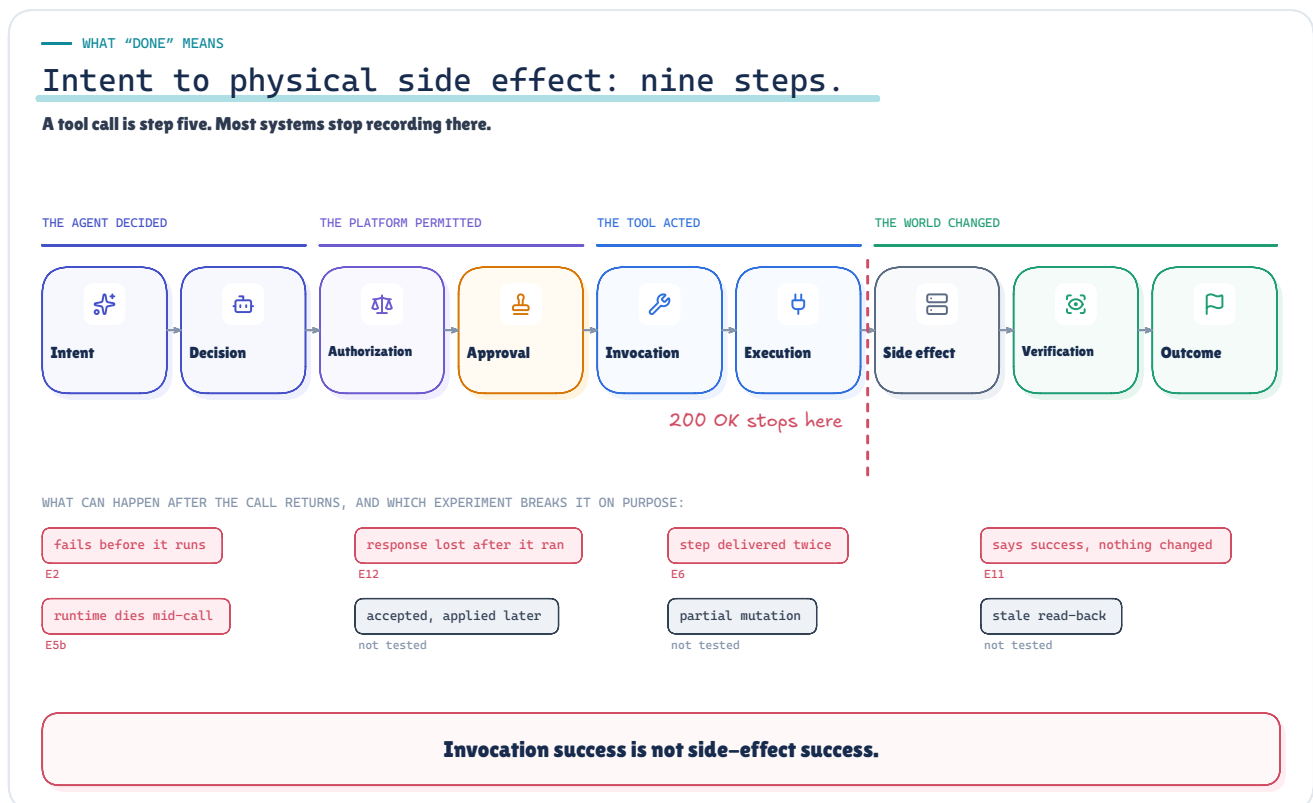


Figure 3. Intent to physical side effect in nine steps. The tool call is step five, and most systems stop recording there.

Architecture: concept figure; experiment tags name the scenarios that inject each failure

1. **Intent.** What triggered the execution and on whose behalf: a monitor alert, invoked under sre-team's delegation.
2. **Decision.** What the model proposed and the structured reason it gave, under an exact model, prompt template and configuration.
3. **Authorization.** Which policy, which version, which rule, which attributes, which obligations.
4. **Approval.** Who was eligible, who decided, and the exact action their decision covers.

5. **Invocation.** Which capability the gateway called, with which arguments, under which tool identity.
6. **Execution.** What the external system did with each request it received.
7. **Side effect.** What actually changed in the world, and how many times.
8. **Verification.** How the platform knows, from which source, observed when.
9. **Outcome.** What the platform concluded, and why.

Steps 1 to 5 happen inside the platform. Steps 6 to 9 happen at or beyond its edge, where failures are ambiguous: a request can be accepted and applied later, succeed after its response is lost, be applied twice, apply partially, or be acknowledged and never applied. A timeout doesn't tell you which of these happened.

Invocation success is not side-effect success.

## 4. The execution lineage

The core entity is not a log record. It is the **execution lineage**: every fact in the chain above, recorded against keys that survive the boundaries between components.

**OUR SYNTHESIS** the correlation model this article uses; the names are the POC's, the structure is the point

| Key                     | One per                             | Created by                               | Survives                                                          |
|-------------------------|-------------------------------------|------------------------------------------|-------------------------------------------------------------------|
| execution_id            | run of the workflow for one trigger | the runtime, at the first step           | process restarts (it is in the checkpoint)                        |
| trace_id                | the same run, for telemetry         | the runtime's tracer                     | process and network boundaries (W3C traceparent <sup>[13]</sup> ) |
| workflow_id             | remediation of one incident         | the orchestration layer                  | re-planning within the incident                                   |
| policy_evaluation_id    | policy decision                     | the policy engine                        | into the approval request and the action                          |
| approval_id             | approval request                    | the approval service                     | into the action's authorization                                   |
| action_id               | decided action                      | the tool gateway, from the action digest | retries and duplicate deliveries; it is the Idempotency-Key       |
| attempt_id              | try at an action                    | the gateway, numbered durably            | everything; .a1 and .a2 are never the same try                    |
| external_transaction_id | what the external system did        | the external system                      | its own records, looked up by idempotency key                     |

Three properties matter more than the names:

- **Each boundary adds an identifier and keeps the ones before it.** The approval request carries the policy evaluation id; the action carries both; each attempt carries the action. Nothing downstream replaces an upstream key.
- **Each identifier is recorded on both sides of the boundary it crosses.** The deployment API stores the idempotency key it received; the gateway stores the transaction id it was given. That is what makes the join a lookup instead of a guess.
- **The keys are recorded durably before the step they describe runs.** An attempt is written down before its request is sent, so a process that dies mid-call leaves a record saying "attempt 1 was in flight".

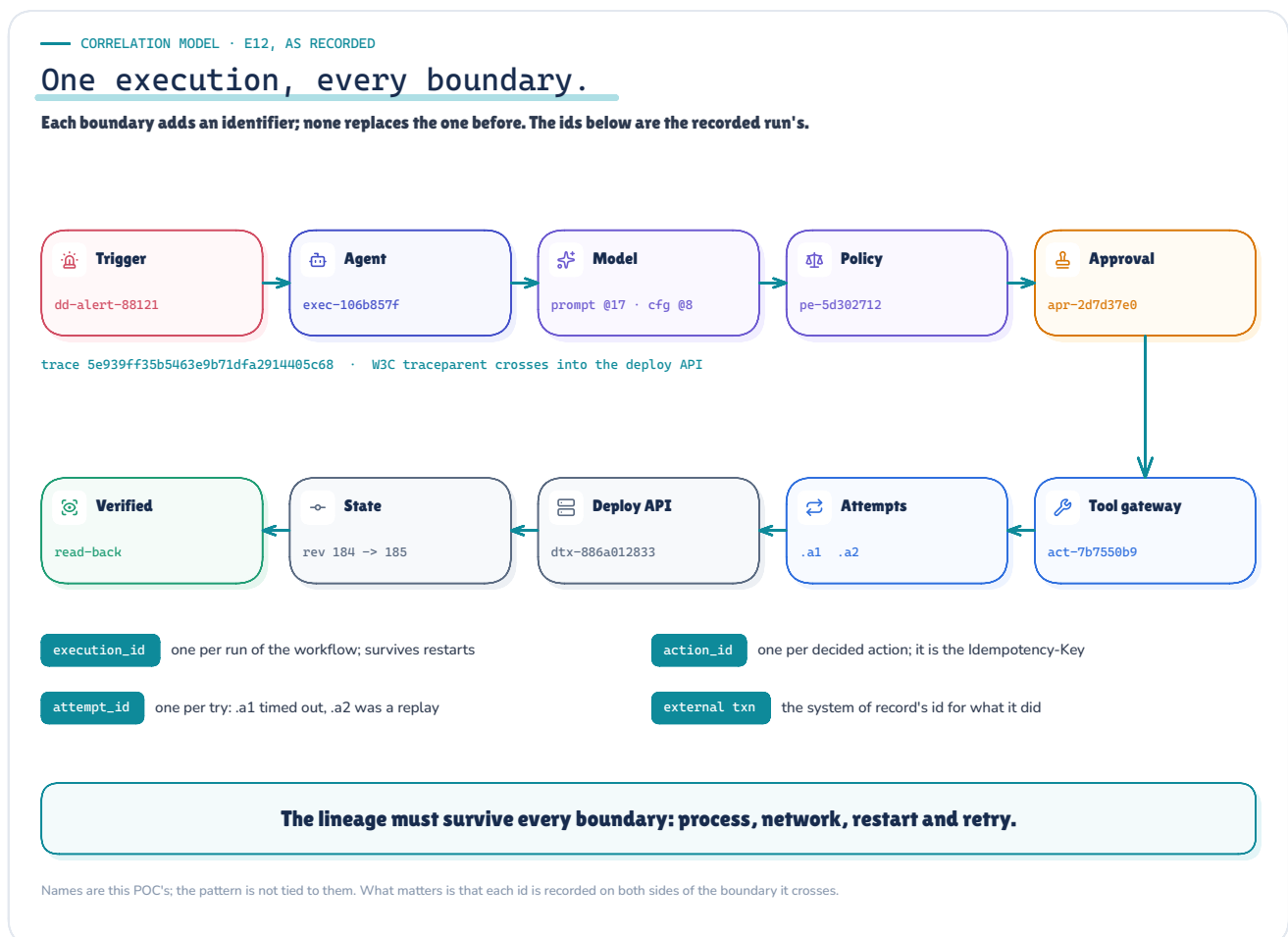


Figure 4. One execution from E12, as recorded: every boundary adds an identifier, and none replaces the one before.

Recorded: every id is E12's, from its evidence · run 2026-09-30-recorded

You should be able to reconstruct an agent action from intent to physical side effect.

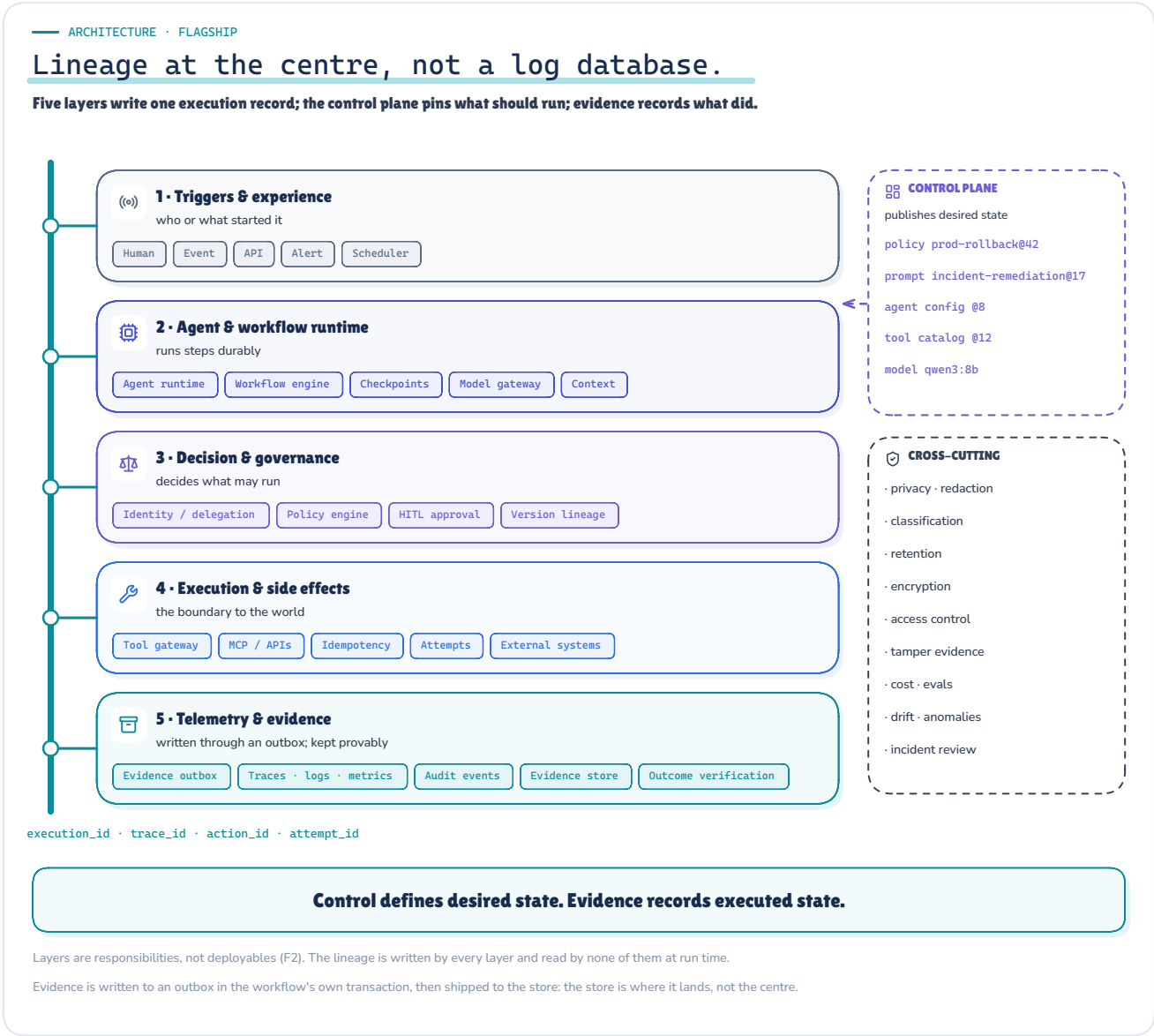
# 5. The layered architecture

ARCHITECTURE

IMPLEMENTED

every layer below exists in the POC in simplified form; production substitutions are named in §26

F2 split an agent monolith into layers, each owning one kind of change. This article reuses that split and asks a different question of it: which facts does each layer contribute to the record, and what holds them together?



2. **Agent and workflow runtime** records the workflow and its steps, checkpoints, and every model call's metadata (F2).
3. **Decision and governance** records identity and delegation, the policy evaluation, approval requests and decisions, and the versions of everything that governed the run (T1, T2, T3, the Control Plane).
4. **Execution and side effects** records each action, each attempt, what the external system said, and what it did (F1, F2).
5. **Telemetry and evidence** holds two records that start from the same events: operational telemetry for the people who run the system, and governance evidence for the people who have to account for it.

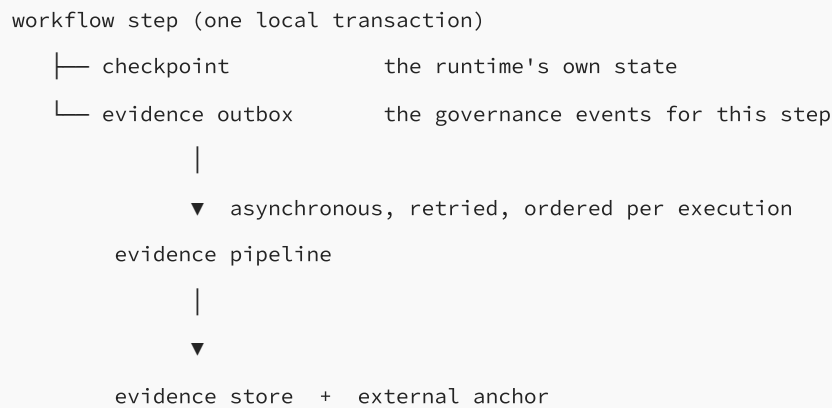
The spine on the left is the point. It isn't a sixth layer or a database. It is the set of keys every layer writes, so that the record can be read end to end afterwards without anybody at run time depending on it.

Control defines desired state. Evidence records executed state.

The control plane (the previous article) publishes desired state: policy `prod-rollback@42`, prompt template `incident-remediation@17`, agent configuration `incident-agent-prod@8`, tool catalog `@12`, model `qwen3:8b`. None of that tells you what a particular execution ran under. A runtime can pin an older version, a rollout can be half-done, an override can be in force. The runtime's evidence has to record what it actually used, with digests, at the moment it used it (§10).

## The evidence outbox

The principle in §4 (record the attempt before the request is sent) raises the obvious question: what happens when the evidence service is unavailable? The answer belongs in the architecture, not in a footnote. Evidence is written to an **outbox in the workflow's own transaction**, alongside the checkpoint, and shipped to the evidence store asynchronously:



The durable intent is recorded locally before the side effect runs, so the side effect can proceed while the evidence service is down; the outbox publishes the events when it recovers, in order, without loss. What the platform must never do is proceed *without* the local record. The POC collapses the outbox and the store into one local SQLite database written in the step's transaction, which gives the same ordering and durability guarantees on one machine; §26 lists what production substitutes.

## 6. Operational telemetry vs governance evidence

Both records start from the same runtime events. They are not the same record, and treating them as one is the mistake that makes "we have logs" feel like an answer. Telemetry *can* be retained immutably and used as audit evidence; it just isn't automatically. Operational pipelines are designed to sample, buffer, drop and expire, and the table below is what changes when a record has to survive an audit instead.

## Telemetry is not automatically evidence.

They start from the same runtime events. They are kept for different people, for different times, under different rules.



Figure 6. Same events, two consumers. The properties are the POC's, with the measured losses from the recorded run.

Architecture + measured: sampling, orphaned spans and retention from the run and config · run 2026-09-30-recorded

|                                      | Operational telemetry                                    | Governance evidence                                                                       |
|--------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Question it answers                  | What happened, where, how fast, how often, at what cost? | Who, on whose authority, under which rules, approved by whom, with what effect, provably? |
| Consumer                             | On-call engineers, SRE                                   | Incident review, security, compliance, audit                                              |
| Schema                               | Open: whatever instrumentation emits                     | Closed: a schema per event type, with an allow-list of fields                             |
| Sampling                             | Normal and often necessary                               | Never                                                                                     |
| Durability                           | Best effort; buffered export                             | Written before the next step; a lost event is a defect                                    |
| Retention (POC config, illustrative) | 7 days for debug logs, 30 for traces and metrics         | 400 days                                                                                  |
| Access                               | Broad within engineering                                 | Restricted, and access is itself logged                                                   |
| Integrity                            | None expected                                            | Tamper-evident, anchored outside the store                                                |
| Content                              | May be rich for debugging, within privacy limits         | Minimal: identifiers, decisions, digests; never raw prompts or secrets                    |

The recorded run measured two of these rows directly. Two crash scenarios SIGKILLED an agent process mid-execution. Those kills left 13 exported spans whose parent span was never exported, because the parent was still open when the process died, and 2 processes' in-memory metrics that were never flushed. The evidence store lost nothing: every event had been committed before the step that followed it, and all 15 of 15 scenario chains verified. And if the run had used head sampling, a 10% ratio would have kept 1 of the 15 target executions' traces and a 1% ratio 0 <sup>[17]</sup>. That is fine for latency dashboards. It is not fine for the record of who changed production.

OpenTelemetry's own guidance points the same way from the other side. The GenAI conventions make prompt and completion content opt-in, mark it sensitive, and suggest that production systems store content externally and put references on spans <sup>[3]</sup>. Telemetry is deliberately not the place for everything.

Operational telemetry is not automatically governance evidence. It tells you what happened; governance evidence tells you whether it was acceptable, controlled and accountable, and lets you prove it later.

## 7. What an agent trace actually needs

Telemetry is still essential, and it should follow the conventions. The POC instruments every process with the OpenTelemetry SDK (1.45.0) and the GenAI semantic conventions. Those

conventions are in *Development* status, not stable, and at the time of the run they had just moved to their own repository; v1.41.1 is the last tagged release that contains them [9] [10]. The POC uses these names:

| Span            | Name                                                 | Kind            | Attributes (all GenAI ones are Development)                                                                                                                                                                                                                  |
|-----------------|------------------------------------------------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Agent execution | invoke_agent incident-agent-prod                     | INTERNAL        | gen_ai.operation.name , gen_ai.agent.name , gen_ai.agent.id , gen_ai.agent.version [4] [7]                                                                                                                                                                   |
| Model call      | chat qwen3:8b                                        | CLIENT          | gen_ai.provider.name (not the deprecated gen_ai.system ) [6] , gen_ai.request.model , gen_ai.response.model , gen_ai.request.temperature , gen_ai.request.seed , gen_ai.usage.input_tokens , gen_ai.usage.output_tokens , gen_ai.response.finish_reasons [1] |
| Tool attempt    | execute_tool deployment.rollback                     | INTERNAL        | gen_ai.tool.name , gen_ai.tool.call.id , gen_ai.tool.type [2]                                                                                                                                                                                                |
| HTTP call       | POST /server POST /v1/deployments/{service}/rollback | CLIENT / SERVER | stable HTTP conventions: http.request.method , http.route , http.response.status_code , url.full , error.type                                                                                                                                                |

Four design decisions matter more than the names.

**Propagate context across every process boundary.** The tool gateway injects W3C `traceparent` into the request to the deployment API, and the API extracts it, so the server span and every log line the API writes during that request carry the agent's trace id [13] [14]. The test suite asserts this against a real HTTP server process.

**Continue the trace after a restart.** The runtime saves the root span's trace and span id in its checkpoint. A restarted process starts its root span as a child of the saved context and adds a span link marked `resumed_from` [15]. The dead process's root span was never exported, so its children are orphans. The trace id still ties them together, and the evidence records the resume explicitly (`workflow.resumed`).

**Correlate logs with traces.** Every application log line written while a span is active carries `trace_id` and `span_id`, which is what the OpenTelemetry logs data model expects [16]. In the POC this is the only difference between the L0 and L1 investigators' log inputs.

**Keep identifiers out of metric labels.** Metrics use low-cardinality attributes only: capability, result, decision, policy version, token type. Execution, action, attempt and trace ids stay on spans and events. The run produced 19 metric series, and check C6 asserts that no metric attribute carries an id. SDKs enforce a cardinality limit, 2000 series per instrument by default, for exactly this reason [21].

What the POC deliberately does *not* put on spans is governance: the policy version, the approver, the delegation, the verified effect. You can put all of these on spans; they are just attributes. But then you have built the evidence record on a pipeline designed to sample, buffer, drop and expire. The L1 layer in the experiment is exactly "standard telemetry", so the measurement shows what that pipeline answers without the evidence semantics.

## 8. Model and decision metadata, not reasoning

The governance primitive for a model call is **decision evidence**, not the model's hidden reasoning. For every call the model gateway records:

| Field                                        | E1's value                                  |
|----------------------------------------------|---------------------------------------------|
| provider, model                              | ollama , qwen3:8b                           |
| model digest, as the model server reports it | sha256:500a1f067a9f...                      |
| prompt template and its digest               | incident-remediation@17                     |
| agent configuration                          | incident-agent-prod@8 (temperature 0)       |
| input digest                                 | sha256 of the exact messages sent           |
| token counts                                 | 576 in, 159 out                             |
| latency, finish reason, replayed or live     | from the call                               |
| structured output                            | deployment.rollback payment-service v4.17.2 |

The proposal also carries a one-to-two-sentence rationale, because the tool schema asks for one explicitly. That is a stated reason the model was asked to produce, stored as a bounded summary (280 characters, classified INTERNAL). It is not a transcript of how the model got there.

Thinking is disabled in the request, and no free-text reasoning is stored anywhere. The evidence schema has no field that could hold a prompt, retrieved context, messages or reasoning, and a unit test asserts that none can be added by accident. Two reasons, one practical and one principled. Practically, the hosted providers don't return raw chain-of-thought anyway: Anthropic returns summarized or encrypted thinking, OpenAI returns reasoning summaries <sup>[47]</sup> <sup>[48]</sup>. OpenTelemetry does define an optional reasoning message part, but capturing it is opt-in <sup>[12]</sup>. In principle, what an investigator needs is what the agent was shown (by digest), what it was asked to do (the template and configuration, by version and digest) and what it decided (the structured output). A narrative of internal deliberation adds retention risk without adding accountability. I found no standard that says "do not store chain-of-thought", so this is the series' position, not a citation.

Useful decision evidence is not raw chain-of-thought.

## 9. Identity lineage

T1 established that five identities stand behind an agent's action: the invoker, the human or group it represents, the agent, the runtime workload, and the credential the tool actually presents. For the record, what matters is that the chain lands in the same execution record as the action.

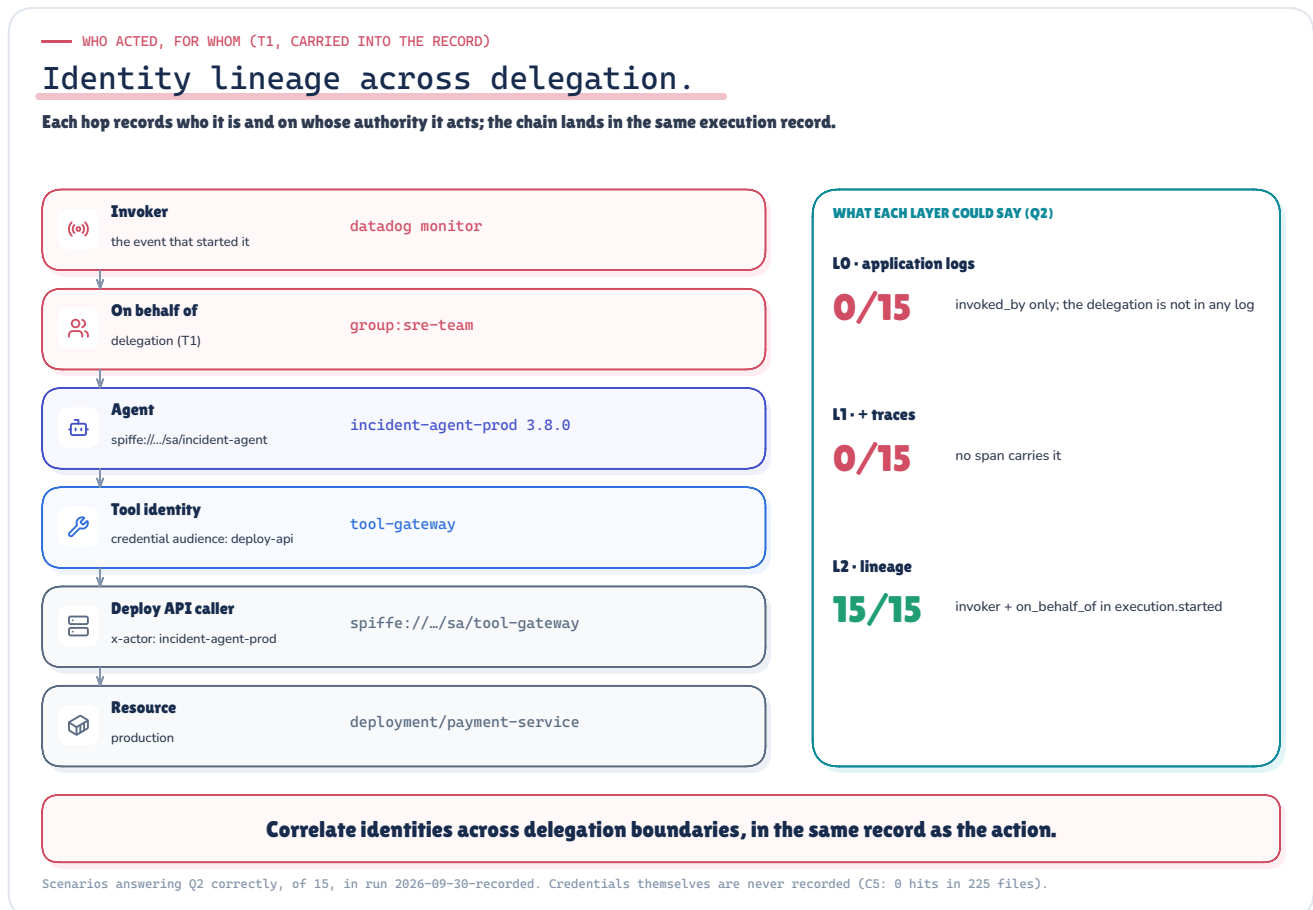


Figure 7. Each hop records who it is and on whose authority it acts. Only the lineage said on whose behalf the agent acted.

Implemented + measured: the delegation chain the POC records; Q2 scores · run 2026-09-30-recorded

In the run, only the lineage said on whose behalf the agent acted: L0 0 of 15, L1 0, L2 15. The agent's log recorded `invoked_by`, as agent logs do. The delegation is a platform concept, and it only exists in the record if the platform writes it there.

The credential itself is never recorded. The deployment API resolves the gateway's bearer token to a workload identity and logs that identity. Check C5 scanned 225 files the run produced (every log, span file, tape and evidence export) for the token and found 0 occurrences. The evidence schema

refuses any value that looks like a bearer token, an API key of that shape or a card number, and the tests exercise all three.

## 10. Policy lineage

The policy engine is deterministic and versioned. Each document is a file with an id and a version; the evaluation returns the id, the version, the sha256 of the exact file evaluated, the matched rule, the obligations and the attributes it read:

```
{
  "policy_evaluation_id": "pe-...",
  "policy_id": "prod-rollback",
  "policy_version": 42,
  "policy_digest": "sha256:24395a0d4331...",
  "decision": "ALLOW_WITH_APPROVAL",
  "rule": "R1-rollback-tier0",
  "obligations": { "approver_roles": ["IncidentCommander", "ServiceOwner"], "quorum": 2,
"expires_in_s": 600, "verify_effect": true },
  "attributes": { "capability": "deployment.rollback", "environment": "production", "service":
"payment-service", "agent": "incident-agent-prod", "severity": "SEV-1" }
}
```

E8 runs the same incident twice, pinned to v41 and v42. Under v41 ( `sha256:6ce10d562f22...` ) the rollback needed quorum 1, with no verification obligation, and got 1 approval. Under v42 ( `sha256:24395a0d4331...` ) it needed quorum 2 and an obligation to verify the effect, and got 2 approvals. The digest is what makes this evidence rather than a label: two teams can both call something "v42".



Figure 8. The control plane says what should run; the evidence says what did, with digests. Only the lineage recorded which configuration produced a proposal.

Recorded + measured: E1, E8a, E8b, E9 pins and digests; Q4 scores · run 2026-09-30-recorded

The honest finding: the application logs answered "which policy and version" in every scenario (15 of 15), because a decent decision log records the bundle revision the way OPA's does. Policy lineage is where good existing practice already works. What the logs lacked was the link from that decision to the approval and the action; the investigator had to join on the incident id and hope there was only one decision.

## 11. Approval lineage

T3 built the durable approval gate. The piece that matters here is the binding. An approval request carries the digest of the exact action it covers: capability, service, environment, target version, incident and policy evaluation id. The approval service signs each decision (HMAC in the POC) over the approval id, that digest, the approver and the decision, and signs only for a person the directory says holds an eligible role. Before the tool gateway lets the action leave the platform, the runtime checks every decision it relies on: the signature is valid, and the digest matches the action about to run.

A decision copied from one action to another fails the check. The test suite does exactly that. It approves a rollback to v4.17.2 and presents the decision for a rollback to v4.16.9: `digest_match` is false. It then rewrites the decision's digest to the new action: `signature_valid` is false. Ineligible approvers are refused before anything is signed, and under v42 the request stays PENDING until both roles have approved.

The evidence records `approval.requested` (id, digest, scope as shown to the approver, eligible roles, quorum, expiry, policy evaluation) and one `approval.decided` per decision (approver, role, decision, reason, time, and the two check results). That answers the question review boards actually ask: not "was it approved" but "who approved *what*".

## 12. Tool and capability lineage

The tool gateway is the execution boundary. Before anything leaves the platform it checks three things. The capability must be in the agent's tool catalog. The call must carry a policy evaluation that permitted it. And if that evaluation required approval, the approvals must be bound to this action's digest.

In E7 a compromised plan asks for `deployment.scale` with zero replicas and then calls the gateway directly, skipping the workflow. The policy engine had already denied the proposal; the gateway refused the direct call too, because the capability is not in catalog `@12` and the call carried no policy evaluation. Both refusals are in the evidence ( `policy.evaluated` DENY, then 1 `gateway.denied` ), and nothing reached the deployment API. This is OWASP's excessive-agency mitigation of complete mediation in the downstream system, made visible <sup>[45]</sup>.

For an authorized action the gateway records `action.authorized` once: the capability, the target, the allow-listed arguments, the catalog version, the tool identity, the credential's audience, the idempotency key, the policy evaluation id, the approval ids and the action digest. It then records every attempt twice. `attempt.started` goes in *before* the request is sent: attempt number, request id, idempotency key, endpoint. `attempt.finished` goes in *after*: result, HTTP status, external transaction id, whether the response was a replay, latency, and the status the tool claimed. Arguments are recorded as the capability's allow-listed fields, never as a raw payload.

## 13. From invocation to physical side effect

`attempt.finished` with `COMMITTED` means the deployment API *said* it rolled back. That is a claim. The governed runtime then reads the world back and records `effect.verified` :

```
{
  "verified": true,
  "expected": { "version": "v4.17.2", "revision_delta": 1 },
  "observed_before": { "version": "v4.18.0", "revision": 184 },
  "observed_after": { "version": "v4.17.2", "revision": 185 },
  "revision_delta": 1,
  "transactions_for_key": ["dtx-b7bec74138"],
  "source": "deploy-api GET /v1/deployments (read-back)",
  "observed_at": "...",
  "health": { "healthy": true, "error_rate": 0.004 }
}
```

— WHAT THE RECORD HOLDS · E12, AS RECORDED

## The execution evidence record.

**16 events for this execution, in a chain it shares with the concurrent one. Decision evidence, not reasoning.**

exec-106b857f · trace 5e939ff35b5463e9... · wf-remediate-inc-4471

**Who**

invoker monitor/payment-service-error-rate  
for group:sre-team  
agent incident-agent-prod 3.8.0

**Model & config**

qwen3:8b sha256:500a1f067a9f...  
incident-remediation@17  
incident-agent-prod@8 · temp 0

**Policy**

prod-rollback@42 ALLOW\_WITH\_APPROVAL  
rule R1-rollback-tier0  
pe-5d302712 · quorum 2

**Approval**

apr-2d7d37e0  
ic.dev · IncidentCommander · APPROVED  
owner.payments · ServiceOwner · APPROVED

**Action & attempts**

act-7b7550b9 = Idempotency-Key  
.a1 TIMEOUT 1,503 ms  
.a2 REPLAYED dtx-886a012833

**Effect & outcome**

revision 184 -> 185 (v4.17.2)  
txns for key: 1 · verified  
outcome MITIGATED

**INTEGRITY**

seq 19 · event\_hash a74bd13db6b7f272b9c6d9d3... · prev\_hash links every event to the one before  
chain head anchored in a witness the runtime cannot rewrite · verify-evidence: intact

Not recorded: the prompt text, retrieved documents, model reasoning, credentials. Their digests are; the schema refuses the rest.

Values from observability\_governance\_poc/runs/2026-09-30-recorded/scenarios/e12a-lost-response/evidence/audit-events.jsonl.

Figure 9. What one execution's record holds: sixteen events in a chain shared with the concurrent execution, grouped. Decision evidence, not reasoning.

Recorded: E12a's evidence events, grouped · run 2026-09-30-recorded

The "before" state is observed at authorization time and checkpointed, so a crash between the two reads doesn't lose it. The deployment API behaves like Kubernetes here: a rollback creates a *new* revision rather than returning to the old number <sup>[26]</sup>. That turns out to be what makes duplicate execution visible (§14).

E11 is the case that matters. The deployment API acknowledges the rollback with `200 {"status": "ROLLED_BACK"}`  and never applies it; the rollout controller silently drops it (the fault is injected). The read-back found the deployment still on v4.18.0 at revision 184, still unhealthy. The governed runtime recorded `EFFECT_NOT_OBSERVED` and escalated. The log line a runtime writes when it trusts the tool said "tool call succeeded, incident mitigated". So in this scenario, and only this one, both L0 and L1 answered "was it mitigated?" wrongly.

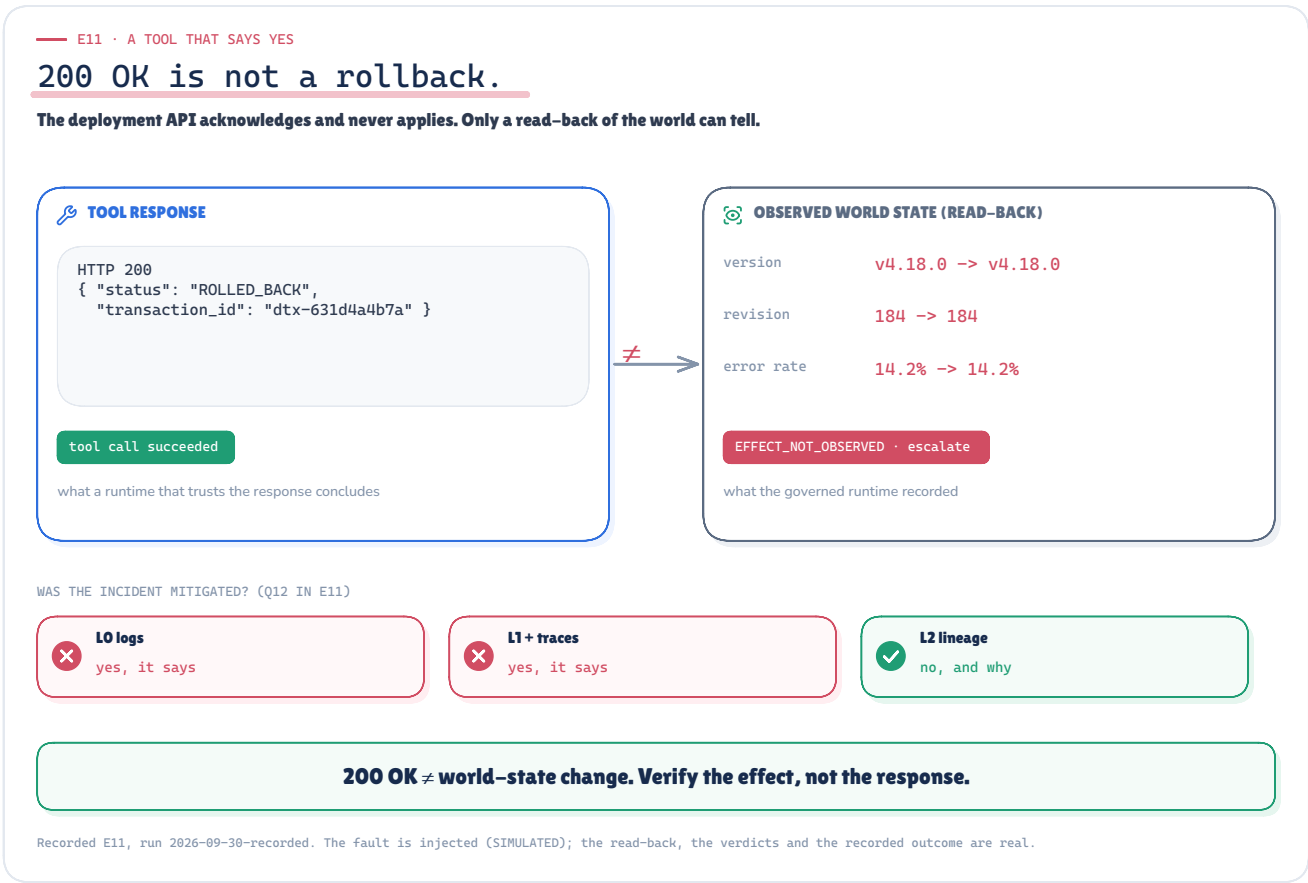


Figure 10. E11: the tool says 200 OK; the world says nothing changed. Only the read-back can tell. Recorded: E11, fault injected (SIMULATED), real read-back and verdicts · run 2026-09-30-recorded

Verify side effects instead of trusting tool responses.

## 14. The unknown-outcome problem

The lost response is the hardest honest case, and it is the flagship experiment. In E12a the deployment API commits the rollback, then withholds its response past the gateway's 1.5-second timeout and closes the connection without sending a byte. The gateway sees a socket timeout after 1,503 ms and cannot know whether production changed. It records that as what it is:

`attempt.finished` with result `TIMEOUT`, error "outcome unknown", not "failed".

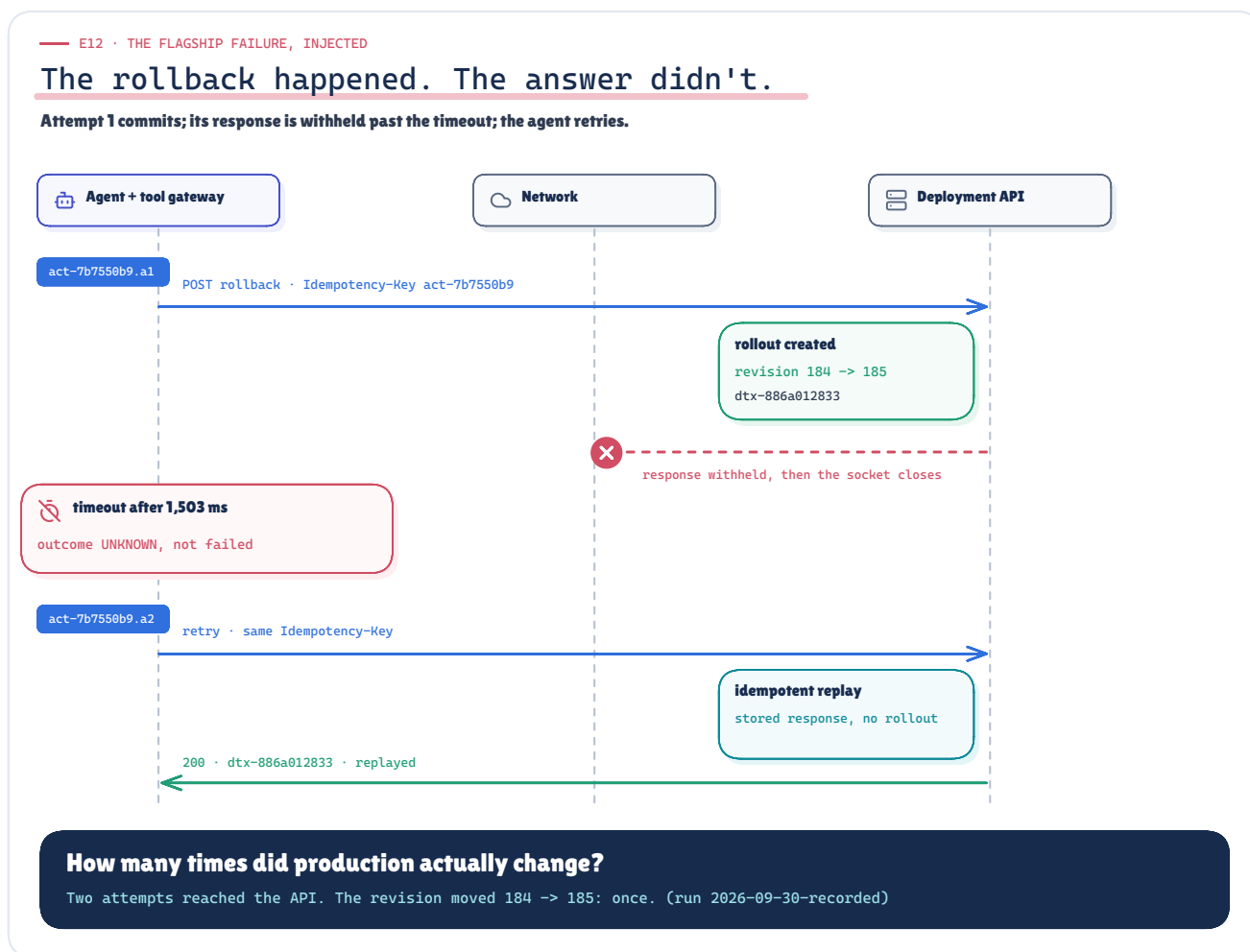


Figure 11. E12a: the rollback commits, its response is withheld past the timeout, and the agent retries with the same key.

Recorded: E12a, one fault injected (SIMULATED), real timeout and retry · run 2026-09-30-recorded

A timeout is retryable, so the gateway retries with the same `Idempotency-Key`, which is the action id. The deployment API finds the key, sees the same request body, and returns the stored response with `Idempotent-Replayed: true`. Nothing is rolled out a second time. This is the pattern in the IETF `httpapi` draft on the `Idempotency-Key` header: first request executes, later requests with the same key and body get the stored result, a key reused with a different body is refused with 422, a key whose first request is still running gets 409 [22] [24] [23]. That draft expired without becoming an RFC; the

behaviour is widely implemented, but it isn't a standard. Plain HTTP doesn't give you this: POST is not idempotent <sup>[25]</sup>.

The recorded run: 2 attempts reached the API, 1 response was dropped, 1 attempt timed out, 1 was an idempotent replay, and production changed 1 time, revision 184 → 185. The lineage links both attempts to one action and one external transaction, `dtx-886a012833`, and the read-back confirms the revision moved by exactly one.

E12b is the control. It's the same fault and the same retry, but the gateway sends no idempotency key. Production changed 2 times: revision 184 → 186. The running *version* was v4.17.2 either way, so a check that looks only at the version passes. In a real cluster that second rollout would have restarted every pod again for nothing. The governed runtime still saw it, because the read-back compares revisions, and recorded `mutations_observed: 2`.

That exposed a flaw in my own verifier. It records the duplicate in the outcome's reason and in `mutations_observed`, but still concludes MITIGATED, because the service is healthy and on the right version. A production verifier should treat "one action, two rollouts" as its own alert even when the service is healthy. That's a design fix I'm recording rather than quietly patching after the run.

— E12a VS E12b · SAME FAULT, SAME RETRY

With the key: once. Without it: twice.

The only difference is whether attempt 2 carries attempt 1's Idempotency-Key.

#### E12a - with key

|                            |   |
|----------------------------|---|
| attempts reaching the API  | 2 |
| timeouts (outcome unknown) | 1 |
| idempotent replays         | 1 |
| responses dropped          | 1 |

#### production changed

1×

revision 184 → 185  
version v4.17.2

the lineage says

1 action · 2 attempts · 1 transaction for the key

the retry was a replay, not a rollout

#### E12b - no key (control)

|                            |   |
|----------------------------|---|
| attempts reaching the API  | 2 |
| timeouts (outcome unknown) | 1 |
| idempotent replays         | 0 |
| responses dropped          | 1 |

#### production changed

2×

revision 184 → 186  
version v4.17.2

the lineage says

1 action · 2 attempts · revision delta 2 · no key to look up

the version looks identical either way

**Idempotency made the retry safe. Lineage made the retry explainable.**

Run 2026-09-30-recorded: truth from the deployment API's own revisions table. The lost response is injected; the timeout and the retries are real.

Figure 12. E12a against E12b: the same fault, the same retry. The only difference is whether attempt 2 carries attempt 1's key.

Measured: E12a vs E12b, truth from the deployment API's own records · run 2026-09-30-recorded

Two different mechanisms are at work, and the article is easy to misread if they blur. **Idempotency** is a runtime control: the key is what stopped attempt 2 from changing production a second time. **Execution lineage** is the record: it is what proves, afterwards, that the two attempts belonged to one action and one external transaction, that the first one's outcome was unknown when it ended, and that the revision moved exactly once. Remove the key and the lineage still tells the truth (E12b's two rollouts are on record); remove the lineage and the key still prevents the duplicate, but nobody can show it did.

Idempotency made the retry safe. Lineage made the retry explainable.

## 15. Retries, crashes and idempotency

Three more scenarios break the execution path in different places.

**E6: the step is delivered twice.** The workflow's tool step completes, but its completion isn't acknowledged, so the queue delivers it again (simulated at-least-once delivery). Both deliveries carry the same action id, and the gateway numbers attempts durably across deliveries: `.a1` committed, `.a2` was a replay. 2 requests reached the API, and production changed 1 time.

**E5a: SIGKILL while waiting for approval.** The request was a committed row in the approval service's database, not state in the process. The supervisor started a new process, which read its checkpoint, recorded `workflow.resumed`, continued the same trace, found the decision and executed once. Result: 2 processes, 1 change.

**E5b: SIGKILL after dispatch.** This is the unknown outcome again, with no process left to retry. The crash point fires after the deployment API's response is in memory and before anything about it is recorded. The new process finds attempt 1 still `IN_FLIGHT` in the checkpoint. It does *not* resend. It asks the deployment API what happened under that idempotency key ( `GET /v1/transactions?idempotency_key=...` ), finds a completed transaction, and records `attempt.reconciled` with result `COMMITTED`. That was 1 reconciliation, 1 attempt reaching the API, and 1 change. Reconciling before retrying is only possible because the key was recorded on both sides.

Give retries and attempts explicit identities, and resolve an unknown outcome by asking, not by repeating.

## 16. Evidence storage and integrity

The POC's evidence store is a SQLite table of append-only events, written in the same local transaction discipline an outbox would use (§5). Every process that writes evidence (both agent runtimes of a scenario, across restarts) appends under `BEGIN IMMEDIATE`, so the chain is one total order even with concurrent writers:

```
event_hash = sha256( prev_hash || canonical(event without its hash fields) )
```

That is tamper *evidence*, not tamper *proof*, and the difference is the useful part of this section. A hash chain detects an edit only if the attacker didn't also recompute every later hash. So when an execution completes, the store writes the chain head (sequence number and hash) to a separate witness file, standing in for WORM storage or a transparency log the runtime can't rewrite. NIST's log-management guide and the tamper-evident logging literature make the same point: integrity digests only help if they are protected somewhere else, and a log needs external witnesses to catch rollback or split views <sup>[33] [42] [43] [41]</sup>.

The run tampered with a copy of E1's evidence four ways:

| Attack                                                                             | Chain check | Anchor check | Detected |
|------------------------------------------------------------------------------------|-------------|--------------|----------|
| T1: change the approver's name on one <code>approval.decided</code> , leave hashes | no          | yes          | yes      |
| T2: the same edit, recompute every later hash                                      | yes         | no           | yes      |
| T3: delete the last three events                                                   | yes         | no           | yes      |
| T4: T2, and rewrite the witness too                                                | yes         | yes          | no       |

("yes" in a check column means that check passed.) T4 is not detected, and that's the honest limit. If the attacker controls the anchor as well as the store, a hash chain proves nothing. The anchor has to be under different control: a different account, WORM storage, a signed transparency log, or a regular export to an audit system the platform can't write to. SQLite is an inspectable stand-in, not a recommendation (§25).

Treat audit evidence differently from debugging logs.

## 17. Privacy, redaction and retention

More telemetry is not automatically better. An agent platform touches credentials, customer data, prompts, retrieved documents, tool arguments and model output. Most of that is exactly what a governance record should *not* keep. GDPR's data-minimisation and storage-limitation principles <sup>[39]</sup> <sup>[40]</sup> and OWASP's sensitive-information-disclosure risk <sup>[46]</sup> both point the same way.

Governance needs evidence, but governance also limits what evidence you are allowed to retain.

What the POC does:

- **Collect by schema, not by habit.** Each evidence event type has an allow-list of payload fields. Anything else is refused at write time, not filtered afterwards.
- **Refuse secrets structurally.** A value that looks like a bearer token, the deploy credential's format or a card number makes the write fail. Tests cover all three.
- **Record digests, not content.** The prompt, the retrieved runbook and the model input are recorded as sha256 digests. The context step records which datasets were read and their classification.

- **Deny restricted data before it reaches the model.** E10's alert annotation asks the agent to attach 50 failed customer transactions. The context step requested `payments.customer_transactions` (RESTRICTED, PCI/PII); the agent's delegation holds INTERNAL scope only, so the request was denied and recorded. A canary string is planted in that dataset. Check C4 searched every model request on the tapes, every log line, every span and every evidence event: 0 occurrences in 225 files. The agent then proposed and executed the rollback anyway, because the restricted data was never needed.
- **Classify and set retention per event type.** Every event type has a retention class in `config/retention.toml`: debug logs 7 days, operational telemetry 30, audit evidence 400. These are illustrative, not legal advice. A test asserts that no event type lacks a class.

What the POC does not do, and production needs: field-level encryption, access control on the evidence store with its own audit trail, separation of duties between who can write evidence and who can read it, and deletion or crypto-shredding for data-subject obligations that apply to identifiers in the record. The EU AI Act, where it applies to a high-risk system, requires automatic logging for traceability and retention of those logs for at least six months (Articles 12, 19 and 26) <sup>[34]</sup> <sup>[36]</sup> <sup>[37]</sup>. Two caveats: most incident agents are not high-risk systems under that Act, and the 2026 Digital Omnibus moved those obligations to December 2027 and August 2028 <sup>[38]</sup>. Nothing here is legal advice. The point is architectural: retention is a property of the record, decided per event type, not a setting on a log bucket.

## 18. Drift and anomaly detection

Agent drift is wider than model drift. The POC records what's needed to see it: model and digest, prompt template, configuration, policy version and decision, tool selection, data access, approval pattern, latency, tokens, outcome. What it measured, with a small probe, is the kind of drift that per-action governance can't see.

D1 asks the same model about ten synthetic incidents: some clearly implicate a release, some are dependency outages, flag flips, traffic spikes or noisy alerts. Each is asked once under agent configuration v8 and once under v9, a routine change of prompt template and sampling temperature.

D1 · 10 INCIDENTS × 2 CONFIGURATIONS · RECORDED

## The behaviour drifted. No rule noticed.

A routine configuration change moved the whole action mix. Policy judges each proposal alone; it cannot see a mix.

config v8 · prompt @17 · temp 0

rollback 7

restart 2

no action 1

config v9 · prompt @18 · temp 0.3

restart 4

diagnostics 6

INCIDENT BY INCIDENT: WHAT EACH CONFIGURATION PROPOSED



- D01 bad release, staging confirmed: v8 rollback, v9 restart
- D03 dependency outage, no release: v8 proposed rolling back a release that had run for days
- 8 of 10 incidents changed action; 3 rollback proposals were denied by policy (services outside it)

**Every action can be individually authorized while the behaviour as a whole drifts.**

reports/drift.json, run 2026-09-30-recorded: one call per incident per configuration (20 calls); a probe, not a detector.

Figure 13. D1: ten incidents, two configurations, one call each. The action mix moved; policy judges each proposal alone and cannot see a mix.

Recorded: D1 drift probe, 10 incidents × 2 configurations, one call each · run 2026-09-30-recorded

|                                   | rollback | restart | diagnostics | no action |
|-----------------------------------|----------|---------|-------------|-----------|
| v8 (template 17, temperature 0)   | 7        | 2       | 0           | 1         |
| v9 (template 18, temperature 0.3) | 0        | 4       | 6           | 0         |

8 of 10 incidents got a different action. Neither configuration is simply "right". v8 proposed rolling back a release that had run for days because an upstream dependency was down. v9 proposed restarting pods for a release that staging had already shown to be bad. The policy engine evaluated every rollback proposal individually. It denied 3 for services outside its rules and would have sent the rest to approval. No rule was broken by the change itself. The planning guide's example assumed drift *toward* rollbacks. The measurement moved the other way, which is a better lesson: you can't predict the direction, only record enough to see it.

This is a probe, not a detector: twenty calls, one per incident per configuration, one seed. It supports one claim: a configuration change can move the whole distribution of actions while every individual action stays within policy. Detecting that in production takes the recorded fields above,

a baseline per action type, and a human who decides what "significant" means for this agent. No single threshold solves anomaly detection, and the evidence record is what makes a distribution computable at all.

Version the policies and configurations that govern execution, and watch the distribution, not just the decision.

## 19. Building the POC

IMPLEMENTED

SIMULATED

`observability_governance_poc/` ; the real/simulated split is itemised in the Real-vs-simulated document

This is a **forensic-reconstruction and failure-injection POC**. It does two separate things:

1. **Reconstruction.** One autonomous production-remediation workflow runs under controlled failures and is read back through three records (application logs, logs plus distributed traces, governed execution lineage). An investigator per record answers thirteen forensic and governance questions, scored against authoritative ground truth from the deployment and approval systems, not against the telemetry itself.
2. **Runtime properties, proven separately.** The same runs exercise the properties that make autonomous actions hard to explain: idempotency, reconciliation after a crash, state verification after a false success, evidence durability across SIGKILL, integrity verification under tampering, and replay without the model.

The claim, that governed execution lineage reconstructs an autonomous change more reliably than fragmented logs, needed a test that could come out the other way. So the POC is built to measure reconstruction, not to demonstrate it.

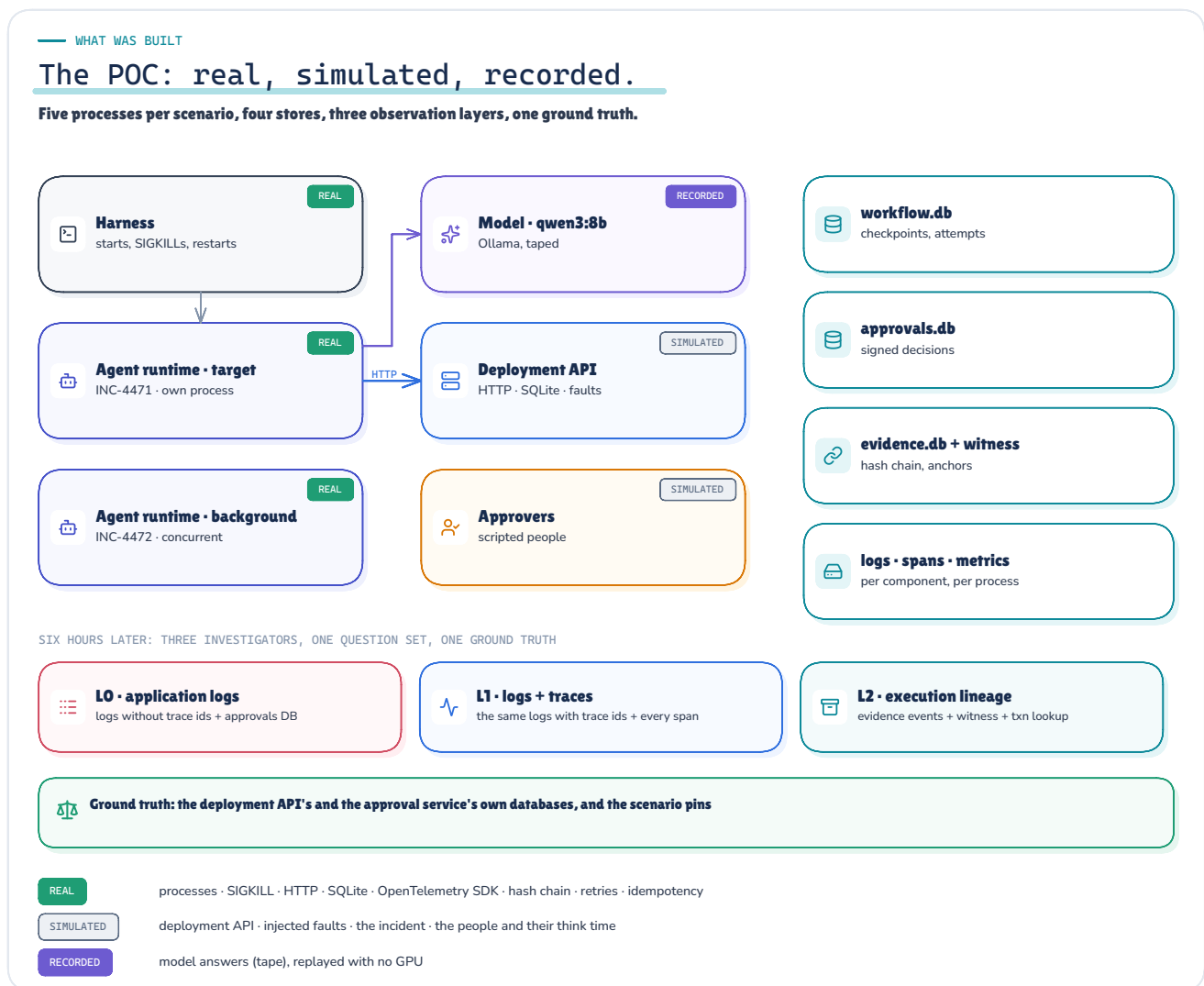


Figure 14. What ran for every scenario: five processes, four stores, three observation layers, and ground truth from the systems of record.

Implemented: the POC's processes and stores; labels as in real-vs-simulated

For every scenario the harness starts five kinds of process:

- **Two agent runtimes**, each a separate Python process running one durable workflow: the target execution (INC-4471, payment-service) and a concurrent background execution (INC-4472, checkout-service) using the same agent, tools, logs and stores. Production doesn't run one execution at a time, and an investigator's joins shouldn't get to pretend it does.
- **The deployment API**, a separate process serving real HTTP on localhost with SQLite as its system of record. It implements the semantics §13–§14 depend on: revisions that only grow, `Idempotency-Key` with stored responses, 409 and 422, bearer-credential identities, and injected faults.
- **The scripted approvers**, a process that answers approval requests through the approval service after a short think time, as the scenario says.

- **The harness**, which supervises the runtimes: a runtime that dies of `SIGKILL` at a named crash point is restarted, the way an orchestrator restarts a pod.

The model is `qwen3:8b` served by Ollama 0.30.11, called through the F2/T3 record/replay tape: the key is the sha256 of the exact request, so a replay answers only if the code asks exactly what it asked while recording. Model calls took a median of 10.9 s on a laptop, partly because the two runtimes queue on one local model.

Across the run: 62 processes, 32 of them agent runtimes; 2 real SIGKILLs; 32 HTTP requests reaching the deployment API; 30 model calls in the scenarios plus 20 in the drift probe; 605 application log lines, 573 spans and 399 evidence events.

## 20. One run, three observation layers

The design choice that matters most is that **each scenario runs once and is observed three ways**. Baseline and governed are not two different systems run separately. They are three views of the same execution:

| Layer                  | What the investigator may read                                                                                                                                                                                                                                                                    |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| L0 • application logs  | Each component's own log ( <code>agent</code> , <code>model-gateway</code> , <code>policy-decisions</code> , <code>tool-gateway</code> , <code>approval-service</code> , <code>deploy-api</code> ) with <code>trace_id</code> / <code>span_id</code> removed, plus the approval service's tables. |
| L1 • logs + traces     | The same log lines with their trace context, plus every span from every process.                                                                                                                                                                                                                  |
| L2 • execution lineage | The evidence events and the witness file, plus the deployment API's transaction lookup by idempotency key.                                                                                                                                                                                        |

Running once removes the usual confound, where the "governed" system also behaves differently. The action path is identical (policy, approvals, checkpoints, retries, idempotency); the records differ, **plus one governed behaviour described next**. It also means the comparison can't be rigged by running the baseline badly.

One behaviour exists *only* to produce evidence: reading the world back after an action. A runtime that trusts the tool declares the outcome from the response; the governed runtime verifies first. To keep L0 honest, the runtime writes the baseline's conclusion to the log at the moment a baseline would ("tool call succeeded, incident mitigated", tagged `profile=baseline` ) and tags the governed verification's lines `profile=governed` . L0 and L1 read the common and baseline lines; L2 doesn't read logs at all. Everything else (idempotency keys, retries, durable checkpoints, the approval gate) is shared by all three. This is the series' platform after F2 and T3, not a strawman.

IMPLEMENTED

`lineage/investigate.py` ; the rules below are in the module's docstring, fixed before the recorded run

The investigators follow written rules:

- Start from the only thing the question gives: *the rollback on INC-4471*.
- A join on an identifier both sides recorded (incident id, request id, trace id, execution id, idempotency key) is a **key join**. A join on time proximity or matching content is a **heuristic join**, counted separately.
- More than one candidate answer is **AMBIGUOUS**, never a guess.
- If the layer doesn't record something, that part of the answer is **None** (the verdict is then *incomplete* or *unanswerable*), never inferred.

The baseline is generous on purpose. Every log that knows the incident id records it. The tool gateway propagates `X-Request-Id` to the deployment API, which logs it, so attempts join exactly. The policy decision log records its bundle revision. The deployment API logs its idempotent replays. A worse baseline would have made the comparison more dramatic and less useful.

**Ground truth** comes only from systems of record and the scenario definition, never from the layers under test ( `lineage/truth.py` ). The deployment API's database says how many requests reached it and how many revisions it created. The approval service's database says who decided what. The scenario pins give the trigger, delegation, agent version, configuration and policy version. The tape holds the model server's reported digest for the pinned model.

The thirteen questions:

| Question |                                                                    | Question |                                                    |
|----------|--------------------------------------------------------------------|----------|----------------------------------------------------|
| Q1       | What triggered the execution?                                      | Q8       | Which capability and arguments reached production? |
| Q2       | Which principal initiated it? (invoker <i>and</i> on whose behalf) | Q9       | How many attempts reached the production API?      |
| Q3       | Which agent and version acted?                                     | Q10      | Did the side effect occur?                         |
| Q4       | Which model and configuration produced the proposal?               | Q11      | How many times did production change?              |
| Q5       | Which policy and version evaluated it?                             | Q12      | Was the incident actually mitigated?               |
| Q6       | Was approval required?                                             | Q13      | Can the evidence's integrity be verified?          |
| Q7       | Who approved or rejected it?                                       |          |                                                    |

Each answer is scored CORRECT, INCOMPLETE (some parts not recorded), UNANSWERABLE (nothing recorded), WRONG (a recorded answer that is false) or AMBIGUOUS.

## 21. The twelve experiments

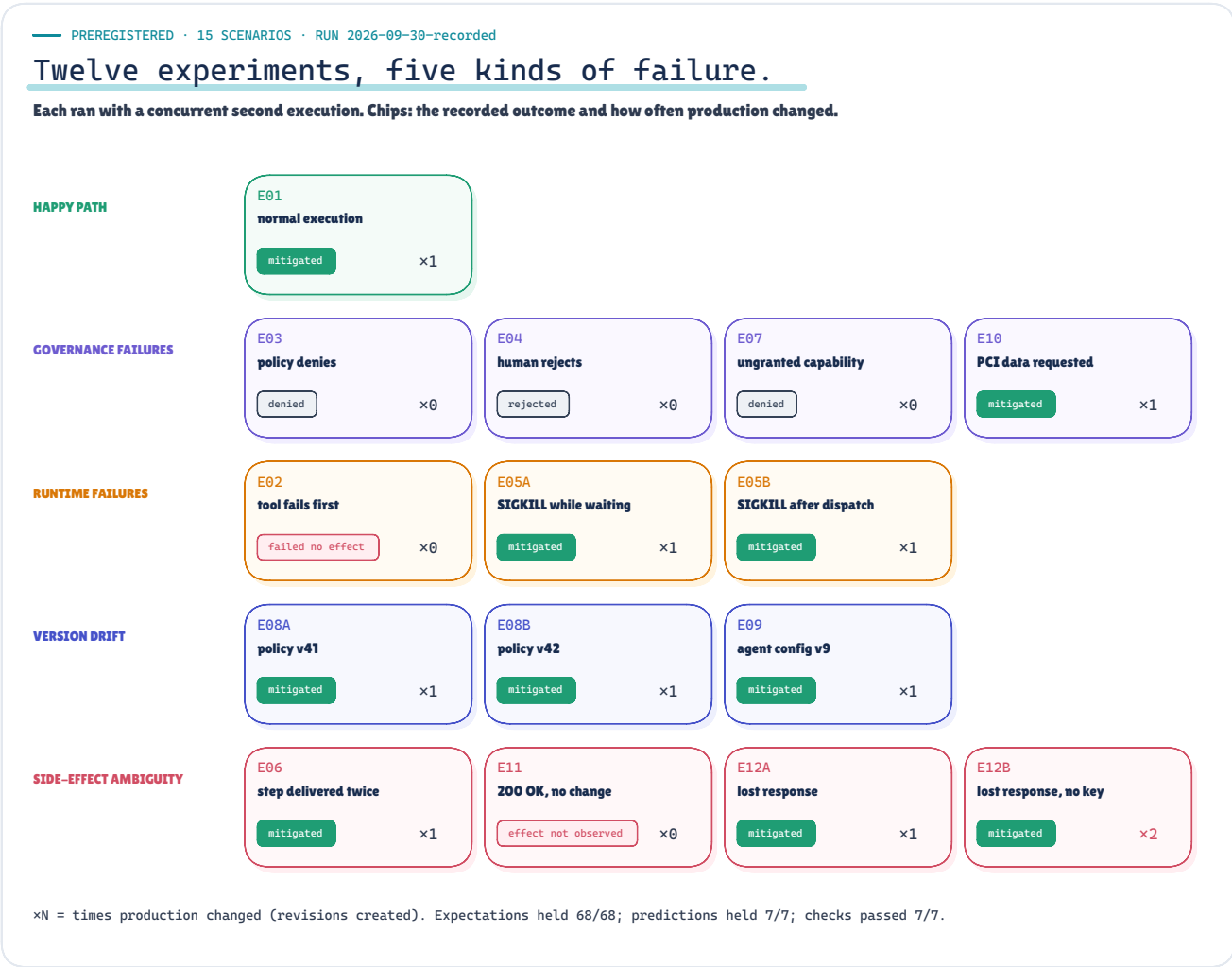


Figure 15. Twelve experiments in fifteen scenarios, grouped by the kind of failure each injects. Chips: the recorded outcome and how often production changed.

Measured: every scenario's recorded outcome and production changes · run 2026-09-30-recorded

| Group                 | Scenario                     | What is injected                                                                     | What it asks                                                       |
|-----------------------|------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| happy path            | e01-success                  | nothing                                                                              | Can one execution be reconstructed end to end?                     |
| governance failures   | e03-policy-denial            | the incident is filed as SEV-3                                                       | A denied proposal: is there proof nothing downstream happened?     |
|                       | e04-human-rejection          | the incident commander rejects                                                       | Is it provable the tool never ran?                                 |
|                       | e07-unexpected-capability    | a compromised plan asks for scale-to-zero and calls the gateway directly (simulated) | Is the attempt and its refusal on record?                          |
|                       | e10-restricted-data          | the alert asks for PCI customer transactions                                         | Is the denial on record, and did any of that data reach the model? |
| runtime failures      | e02-tool-failure             | the deployment API returns 503 before committing, three times                        | Does the record say production did not change?                     |
|                       | e05a-crash-awaiting-approval | real SIGKILL while waiting for approval                                              | Is the approval kept, and is it still one story?                   |
|                       | e05b-crash-after-dispatch    | real SIGKILL after the response arrived, before it was recorded                      | Can the runtime find out without doing it again?                   |
| version drift         | e08a/e08b-policy-v41/v42     | the same incident under two policy versions                                          | Which version governed, with what obligations?                     |
|                       | e09-config-v9                | agent configuration v9 (template 18, temperature 0.3)                                | Does the record say which configuration produced the proposal?     |
| side-effect ambiguity | e06-duplicate-delivery       | the tool step is delivered twice                                                     | Does production change twice?                                      |
|                       | e11-false-success            | the API acknowledges and never applies                                               | Who notices?                                                       |
|                       | e12a-lost-response           | the API commits, then the response is lost                                           | How many times did production change, and can we prove it?         |
|                       | e12b-lost-response-no-key    | the same, with no idempotency key (control)                                          | What does the same retry do without the key?                       |

Before the run I preregistered 68 per-scenario expectations (outcome, production changes, attempts, and so on) and 7 predictions about the reconstruction (for example, "L0 reports the wrong final outcome in e11"). The preregistration file's digest, `sha256:08cc152fc873...`, is frozen in the run manifest, along with the digests of every config file and every source file.

## 22. The recorded proof run

**RECORDED**

`observability_governance_poc/runs/2026-09-30-recorded/`

The published run is `2026-09-30-recorded`. It follows F2's proof discipline:

- **Named and immutable.** The harness refuses to overwrite an existing run directory. The directory holds `manifest.json` (frozen input and code digests), `environment.json`, per-scenario folders (spec, logs, spans, metrics, evidence export and verification, world before/after, the deployment API's request log, truth, reconstruction, result, process logs, model tapes), run-level reports, `facts.json` and `checks.json`.
- **Checked.** 7 of 7 checks passed: every scenario completed, every chain verified, tampering was detected, the canary and the credential appear nowhere, no metric carries an id, and every scenario has exactly one target execution.
- **Replayable without a model.** `lineage run replay 2026-09-30-recorded` reran all 15 scenarios with Ollama unreachable. It served 60 model answers from the tapes with 0 misses, and every outcome, verdict and evidence payload was identical apart from timestamps and latencies (`reports/replay-comparison.json` : identical = yes). The drift probe replayed identically too (yes).
- **Browsable.** The Lab Console shows every scenario through every layer: input, output, the thirteen answers against truth, every recorded step and the data lineage. It opens on the lost-response case, and its Method page states the experiment contract.
- **Tested.** The POC's test suite passed 31 of 31 with the model unreachable. That includes end-to-end scenario tests that start real processes, SIGKILL them, and assert the flagship properties against fresh runs.
- **Honest about its history.** The recording was made twice. The first recording's replay found that the deployment API's transaction ids included the requests' arrival order, which differs between concurrent runs. I made the ids depend only on the per-attempt request id and re-recorded. Every scenario's outcome, change count, attempt count, process count and verdicts were identical in both recordings; the check and the before/after summaries are in `docs/proof-run.md`.

Every measured number in this article is a token resolved from `facts.json` (or from `docs/derived-facts.json`, computed from the same run) at build time. A number that isn't in the run can't appear in the text: an unknown token fails the build.

## 23. What the numbers showed

### MEASURED

`reports/comparison.json`, `reports/results.json`, `facts.json` of run 2026-09-30-recorded

What the POC actually proves, in order of strength: E11 shows that a tool's success response is not evidence of a physical outcome; E12a against E12b is a controlled comparison (same fault, same retry, key or no key: 1 change against 2); E5a/E5b exercise real SIGKILL and durable recovery; the tamper experiment tests the integrity design and finds its limit; E10 tests restricted data with a planted canary; the replay shows the run reproduces without the model. The reconstruction scorecard in §23.2 is supporting evidence, a coverage test, not the headline.

## 23.1 Every scenario ran as preregistered

All 15 scenarios ran to completion, and 68 of 68 preregistered expectations held (missed: none). Production changed exactly once in every scenario that should have changed it, and zero times in every scenario that shouldn't, with one designed exception: the E12b control, which changed it 2 times. The evidence chain verified in 15 of 15 scenarios.

| Scenario                    | Recorded outcome    | Production changes | Attempts reaching the API | Agent processes |
|-----------------------------|---------------------|--------------------|---------------------------|-----------------|
| e01 normal                  | MITIGATED           | 1                  | 1                         | 1               |
| e02 tool fails first        | FAILED_NO_EFFECT    | 0                  | 3                         | 1               |
| e03 policy denies           | DENIED              | 0                  | 0                         | 1               |
| e04 human rejects           | REJECTED            | 0                  | 0                         | 1               |
| e05a SIGKILL while waiting  | MITIGATED           | 1                  | 1                         | 2               |
| e05b SIGKILL after dispatch | MITIGATED           | 1                  | 1                         | 2               |
| e06 step delivered twice    | MITIGATED           | 1                  | 2                         | 1               |
| e07 ungranted capability    | DENIED              | 0                  | 0                         | 1               |
| e08a policy v41             | MITIGATED           | 1                  | 1                         | 1               |
| e08b policy v42             | MITIGATED           | 1                  | 1                         | 1               |
| e09 agent config v9         | MITIGATED           | 1                  | 1                         | 1               |
| e10 PCI data requested      | MITIGATED           | 1                  | 1                         | 1               |
| e11 200 OK, no change       | EFFECT_NOT_OBSERVED | 0                  | 1                         | 1               |
| e12a lost response          | MITIGATED           | 1                  | 2                         | 1               |
| e12b lost response, no key  | MITIGATED           | 2                  | 2                         | 1               |

A few of these are worth reading closely. E3's policy denied the proposal (decision DENY, severity SEV-3), and no approval, attempt or request followed. E2's three 503s produced 3 requests and 0 changes; the read-back confirmed the deployment still on v4.18.0, and the outcome is `FAILED_NO_EFFECT` rather than a bare "failed". E9 is subtler than the drift probe (\$18): with the runbook in context, configuration v9 still proposed `deployment.rollback payment-service v4.17.2`. The evidence recorded `incident-agent-prod@9` and `incident-remediation@18` at temperature 0.3, so an investigator can tell the two configurations' decisions apart even though they agreed here.

23.2 Reconstruction: logs, traces and lineage

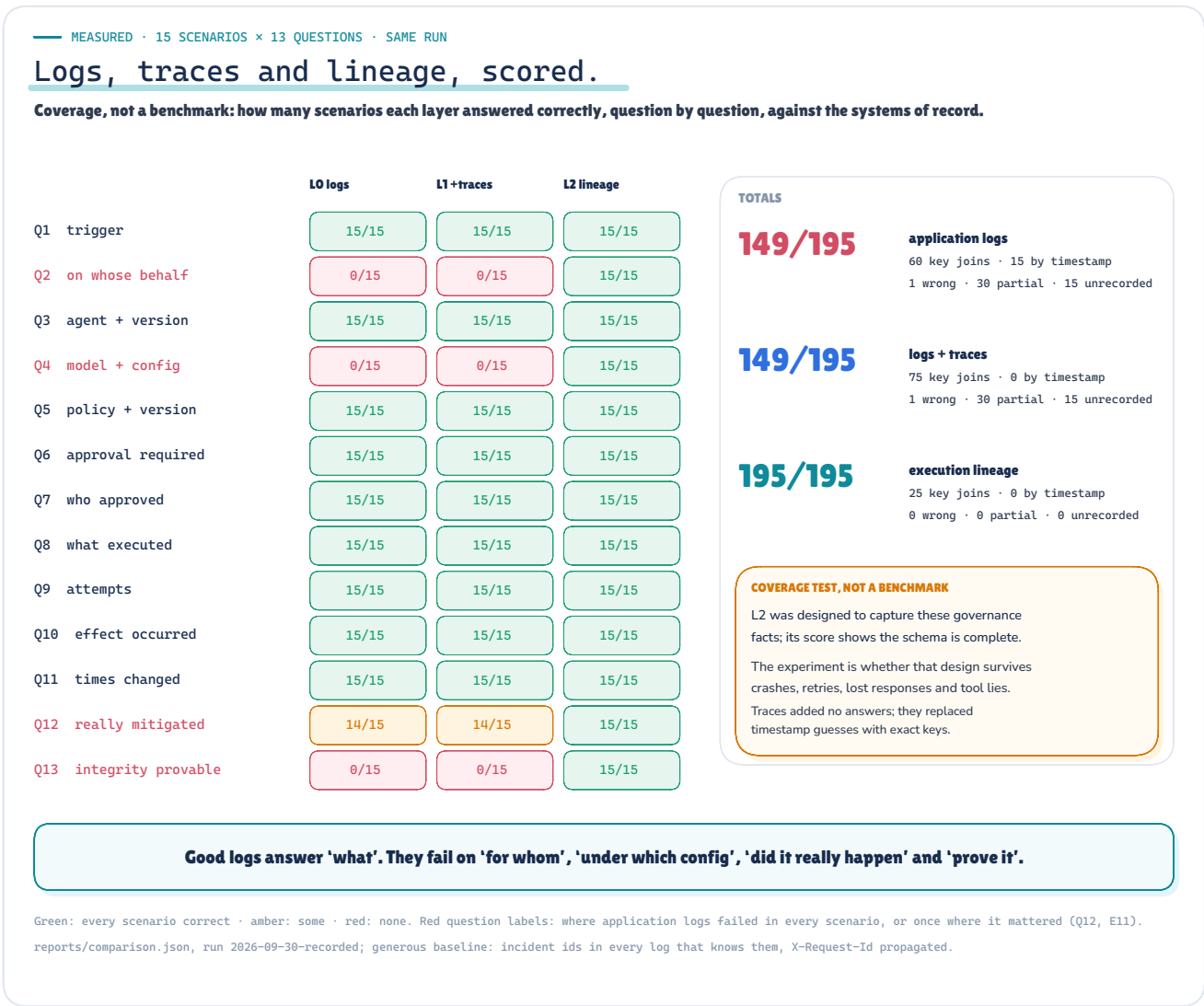


Figure 16. Fifteen scenarios × thirteen questions × three layers, scored against the systems of record.  
Measured: 15 scenarios × 13 questions × 3 layers · run 2026-09-30-recorded

|                        | Correct | Wrong | Partial | Not recorded | Key joins | Timestamp joins | Most sources in one scenario |
|------------------------|---------|-------|---------|--------------|-----------|-----------------|------------------------------|
| L0 · application logs  | 149/195 | 1     | 30      | 15           | 60        | 15              | 6                            |
| L1 · logs + traces     | 149/195 | 1     | 30      | 15           | 75        | 0               | 6                            |
| L2 · execution lineage | 195/195 | 0     | 0       | 0            | 25        | 0               | 3                            |

Read this table carefully, because it is easy to over-read.

**The logs were good.** Every question about *what happened* (trigger, agent, policy version, approval, capability, attempts, whether and how often production changed) was answered correctly in every scenario from logs alone. That includes the lost response: the tool gateway logged attempt 1 as a timeout and attempt 2 as a success, the deployment API logged one "rollout created" and one "idempotent replay", and the request ids joined them. A well-run service estate gets further than the "we have logs" joke suggests.

**The logs failed on four questions, and they are the governance ones.** On whose behalf (0/15); under which prompt template, configuration and model digest (0/15); whether the incident was *really* mitigated (14/15, wrong exactly in the false-success case); and whether the record can be trusted (0/15). Nobody logs the delegation or the prompt version unless someone decided they belong in the record, and an ordinary, mutable log cannot, by itself, prove its own history has not been altered.

**Traces added no answers.** L1 answered exactly what L0 did. What traces changed was the method: 0 timestamp joins instead of 15, because the model gateway's line, which carries no incident id, could be found by trace id instead of by time. In this run the timestamp join was always right. The executions were seconds apart and on different services, so the guess never had a chance to go wrong. The value of trace context is that it doesn't depend on that luck.

**L2's 13/13 is a coverage test, not a benchmark.** It is partly by construction. Its schema was written around these questions, so a perfect score mostly shows the implementation is complete. The informative parts are what it had to survive to stay correct: a restart mid-approval, a restart mid-call, a lost response, a duplicate delivery, a tool that lied, and a concurrent execution writing to the same chain. And it did this with fewer joins than the logs (25 key joins in total, at most 3 sources per scenario, against 60 and 6).

### 23.3 The flagship, read back

The flagship scenario reads the same way from its evidence alone. This is `lineage inspect e12a-lost-response`, drawn (and, row by row with each layer's answers, in the Lab Console):

— E12 · READ BACK FROM 16 EVIDENCE EVENTS

## Six hours later, the story in order.

Wall-clock offsets of the recorded run. The approvers are scripted, so the human wait is seconds, not minutes.

|           |                     |                                                                                   |
|-----------|---------------------|-----------------------------------------------------------------------------------|
| T+ 0.000s | execution.started   | INC-4471 SEV-1 from datadog:monitor/payment-service-error-rate for group:sre-team |
| T+ 0.000s | context.accessed    | runbooks.payment-service ALLOW                                                    |
| T+ 0.000s | context.accessed    | deployments.history ALLOW                                                         |
| T+ 9.898s | model.invoked       | qwen3:8b · incident-remediation@17 · incident-agent-prod@8                        |
| T+ 9.899s | decision.proposed   | deployment.rollback payment-service v4.17.2                                       |
| T+ 9.902s | policy.evaluated    | prod-rollback@42 ALLOW_WITH_APPROVAL                                              |
| T+ 9.906s | approval.requested  | apr-2d7d37e0 quorum 2                                                             |
| T+10.857s | approval.decided    | ic.dev APPROVED                                                                   |
| T+10.857s | approval.decided    | owner.payments APPROVED                                                           |
| T+10.863s | action.authorized   | act-7b7550b9 key=act-7b7550b9                                                     |
| T+10.864s | attempt.started     | attempt 1                                                                         |
| T+12.369s | attempt.finished    | attempt 1 TIMEOUT                                                                 |
| T+12.577s | attempt.started     | attempt 2                                                                         |
| T+12.584s | attempt.finished    | attempt 2 REPLAYED dtx-886a012833                                                 |
| T+12.591s | effect.verified     | revision 184 -> 185, v4.17.2, verified=True                                       |
| T+12.591s | execution.completed | MITIGATED                                                                         |

**Make the production story reconstructable after the runtime is gone.**

reports/timeline.json, run 2026-09-30-recorded; 'uv run lineage inspect e12a-lost-response' prints the same from the evidence.

Figure 17. E12a read back from its sixteen evidence events, in order, with the recorded run's wall-clock offsets.

Recorded: E12a's evidence events in order, wall-clock offsets · run 2026-09-30-recorded

It is 16 events for this execution. The model took 9.9 s; the scripted approvers 0.9 s (a real incident commander takes minutes); the gateway waited 1,503 ms for a response that never came, and retried 208 ms later. The execution ended 12.6 s after it started. A reviewer six hours later, or six months later, gets the whole causal chain from one query on one key.

## 23.4 What telemetry lost

MEASURED

reports/telemetry.json

The crash scenarios measured what a SIGKILL does to each record:

|                                       | E5a · killed while waiting                                           | E5b · killed after dispatch                                                  |
|---------------------------------------|----------------------------------------------------------------------|------------------------------------------------------------------------------|
| spans exported whose parent never was | 5                                                                    | 8                                                                            |
| agent metric dumps lost               | 1                                                                    | 1                                                                            |
| evidence events lost                  | none: every event was committed before the next step; chain verified | none: the in-flight attempt was on record and was reconciled; chain verified |

The SDK exported each span as it ended, which is the most durable a telemetry exporter gets. The spans still open when the process died (the root span and the current step) were never exported, so their children show up with parent ids that point nowhere. The resumed process's spans continued the trace, so a trace viewer shows one trace with a hole in it. The evidence shows `workflow.resumed` with the previous pid and the in-flight attempt. Neither telemetry loss is a bug; that is how telemetry pipelines are designed to behave. It is the reason the evidence can't live there.

Head sampling would have removed most of the flagship's telemetry before anyone looked. Applying OpenTelemetry's trace-id ratio sampler to the fifteen recorded target trace ids keeps 4 at 25%, 1 at 10% and 0 at 1% <sup>[17]</sup> <sup>[18]</sup>. The current specification deprecates that sampler in favour of a probability sampler with the same effect <sup>[19]</sup>. Tail sampling can keep "interesting" traces <sup>[20]</sup>, but it decides what was interesting from latency and errors, and E11's false success had no error in any span.

## 24. What surprised me

**Good logs did better than the thesis needed.** I expected L0 to be wrong about the lost response. It wasn't: the deployment API logged the idempotent replay, the gateway propagated request ids, and the counts came out right. That changed the argument. The article isn't "logs can't reconstruct an agent's action"; it's "logs reconstruct what happened, and are silent or wrong on the questions governance actually asks".

**Traces changed the method, not the answers.** I had assumed OpenTelemetry would close part of the gap. It closed the correlation gap completely and the governance gap not at all, because a span carries what the instrumentation put on it, and standard instrumentation doesn't know about delegation, policy versions or verified effects.

**The duplicate was invisible to the obvious check.** In E12b both rollouts targeted v4.17.2, so the running version was right after both. Only the revision counter, or the deployment API's own request log, showed that production had changed twice. The verifier caught it; my first verifier also marked the outcome MITIGATED, which hides it (§14).

**The drift ran the other way.** The plan for this article assumed a configuration change would *increase* rollbacks. It removed them. The same probe shows why direction doesn't matter: every proposal was individually evaluated and most would have gone to a human, and still the behaviour as a whole changed completely (§18).

**Replay caught a non-determinism the tests didn't.** The first recording's replay was identical in every outcome and verdict but not in the evidence. The deployment API's transaction ids depended on the order in which two concurrent executions' requests arrived. The tests passed either way. Only a byte-level comparison of two runs' evidence showed it, which is an argument for replay comparison as a CI check.

## 25. What the POC did not prove

**LIMITATION** read before citing any number above

- **Not a real cluster or a real deployment system.** The deployment API is a simulation with the semantics that matter here (growing revisions, idempotency keys, stored responses). Real controllers are asynchronous, can partially apply a rollout, and have eventual-consistency windows that make a read-back stale. E11's fault is one kind of lie; "accepted, applied later" and "partial mutation" were named and not tested.
- **Not a regulated audit system.** The evidence store is SQLite with a hash chain and a file standing in for an external witness. That shows the properties: append-only, schema-bound, anchored. It is not a recommendation for SQLite, and T4 shows exactly what it cannot survive: an attacker who controls the anchor too.
- **Not a SIEM, not a universal schema.** The event types and the correlation keys are this POC's. The claim is about the structure (keys that survive boundaries, recorded on both sides, written before the step), not about these names.
- **One incident, one service, one concurrent neighbour.** Two executions on different services, seconds apart, is a gentle test for timestamp joins. Busier estates, the same incident triggering twice, or two agents acting on one deployment would make L0's heuristics fail; this run doesn't show that, it only argues it.
- **A generous baseline.** The logs carried incident ids and propagated request ids. Many real estates don't, and would do worse. Some log prompt versions and would do better on Q4. The comparison measures one reasonable logging practice, not all of them.
- **L2's perfect score is by construction.** Its schema was designed around the thirteen questions. The measurement shows it is complete and survives the injected failures, not that it would answer questions nobody designed it for.
- **Scripted people, compressed time.** Approvers answer in under a second. Approval latency, fatigue and human error are not measured.

- **A small model, one seed.** qwen3:8b at temperature 0 (0.3 in v9) on a laptop. The drift probe is twenty calls; it shows a distribution can move, not how often it does.
- **Not legal compliance.** Retention periods are illustrative, and nothing here establishes compliance with the EU AI Act, GDPR or any other regime.
- **Immediate read-back only.** The verifier reads the deployment once, straight after the call. The next experiment to add is "accepted now, applied later": an API that acknowledges and applies after a delay, to test verification under eventual consistency (bounded re-reads, and when to give up and escalate).
- **Not an argument to store every prompt.** The POC stores digests; §17 argues for less content, not more.

## 26. Production design decisions

### OUR SYNTHESIS

what I would build, and what I would substitute for the POC's stand-ins

| Decision                       | POC                                                                                                               | Production                                                                                                                                                            |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Correlation keys               | <code>execution_id</code> , <code>action_id</code> (= idempotency key), <code>attempt_id</code> , external txn id | Keep the structure; use time-ordered unique ids (UUIDv7 or ULID); propagate them in W3C baggage or explicit headers as well as trace context                          |
| Where evidence is written      | the component that knows the fact, before the step that follows                                                   | Same, through a local durable outbox (the workflow's own transaction) shipped to the evidence service, so a network partition doesn't block or lose evidence          |
| Evidence store                 | SQLite, append-only, hash chain                                                                                   | An append-only store with retention locks (object storage with WORM/object lock, or a ledger database); per-tenant chains; periodic signed checkpoints                |
| Anchor / witness               | a file                                                                                                            | A transparency log, a separate cloud account's WORM bucket, or a signed export to the organisation's audit platform; never writable by the platform's own credentials |
| Telemetry                      | OTel SDK, JSONL exporter                                                                                          | OTel SDK to a collector; tail sampling for telemetry only; GenAI conventions pinned to a version, since they are still Development                                    |
| Governance attributes on spans | none (measured baseline)                                                                                          | A few references on spans (execution id, action id, attempt id) so traces link to evidence; the facts stay in the evidence                                            |
| Side-effect verification       | read-back of revision, version, health, and the txn lookup by key                                                 | Read-back against the system of record with bounded retries for eventual consistency; treat "one action, more than one mutation" as an alert even when healthy        |
| Unknown outcomes               | retry with the same key; after a crash, reconcile before retrying                                                 | Same, and require every production tool to support either idempotency keys or an outcome lookup; tools with neither need a human in the loop for retries              |
| Model evidence                 | model, digest, template and config versions and digests, input digest, tokens, structured output                  | Same; store prompts and retrieved context only in a separate, access-controlled, short-retention store if at all, referenced by digest                                |
| Privacy                        | schema allow-list, credential and card patterns refused, digests, data-scope check                                | Add classification-driven field encryption, tokenisation of personal identifiers, access logging on the evidence store, and crypto-shredding for deletion obligations |
| Drift                          | fields recorded; a probe                                                                                          | Per-agent baselines of action mix, data access, approval rate and outcome by configuration version; alert on shifts after any config or model change                  |

## 27. Reconstructing the SEV-1

Back to 20:09 and the security review. Every answer below comes from the flagship scenario's evidence and the deployment API's transaction lookup: one key, `exec-106b857f`.

| Question                                | Answer from the lineage                                                                                                                                                                                                                                      |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Who initiated the rollback?             | <code>datadog:monitor/payment-service-error-rate</code> (alert dd-alert-88121)                                                                                                                                                                               |
| On whose authority?                     | <code>group:sre-team</code> , by delegation                                                                                                                                                                                                                  |
| Which agent?                            | <code>incident-agent-prod</code> 3.8.0, workload <code>spiffe://.../sa/incident-agent</code>                                                                                                                                                                 |
| Which model and configuration?          | <code>qwen3:8b</code> ( <code>sha256:500a1f067a9f...</code> ), <code>incident-remediation@17</code> , <code>incident-agent-prod@8</code>                                                                                                                     |
| Which policy, which version?            | <code>prod-rollback@42</code> , rule <code>R1-rollback-tier0</code> , <code>ALLOW_WITH_APPROVAL</code> , quorum 2, verify effect                                                                                                                             |
| Who approved, what exactly?             | <code>ic.dev</code> (IncidentCommander) and <code>owner.payments</code> (ServiceOwner), both on the action digest for <i>rollback payment-service to v4.17.2 in production for INC-4471 under that policy evaluation</i> ; signatures valid, digests matched |
| What executed?                          | <code>deployment.rollback</code> <code>payment-service</code> → v4.17.2, action <code>act-7b7550b9</code> , via the tool gateway's workload identity with audience <code>deploy-api</code>                                                                   |
| How many attempts?                      | 2: <code>.a1</code> timed out after 1,503 ms (outcome unknown), <code>.a2</code> was an idempotent replay of <code>dtx-886a012833</code>                                                                                                                     |
| Did production change, and how often?   | Yes, 1 time: revision 184 → 185, v4.18.0 → v4.17.2, one transaction for the key                                                                                                                                                                              |
| What was the outcome?                   | MITIGATED, verified by read-back: healthy, error rate back to baseline                                                                                                                                                                                       |
| Can we prove the record wasn't altered? | The chain verifies and matches the external anchor                                                                                                                                                                                                           |

That's the whole review. It needed one key, not ten systems.

## 28. The architecture we are left with

### The claim, precisely

The POC does not prove that "execution lineage beats logs". Good logs reconstructed most operational facts.

It shows that production governance requires additional semantics (delegation, configuration lineage, action-bound approvals, verified effects and evidence integrity) and that those semantics must remain reconstructable across crashes, retries and ambiguous side effects.

Runtime controls such as idempotency and reconciliation make the action safe. Execution lineage makes the action explainable and provable afterwards.

## Four floors and nine rules

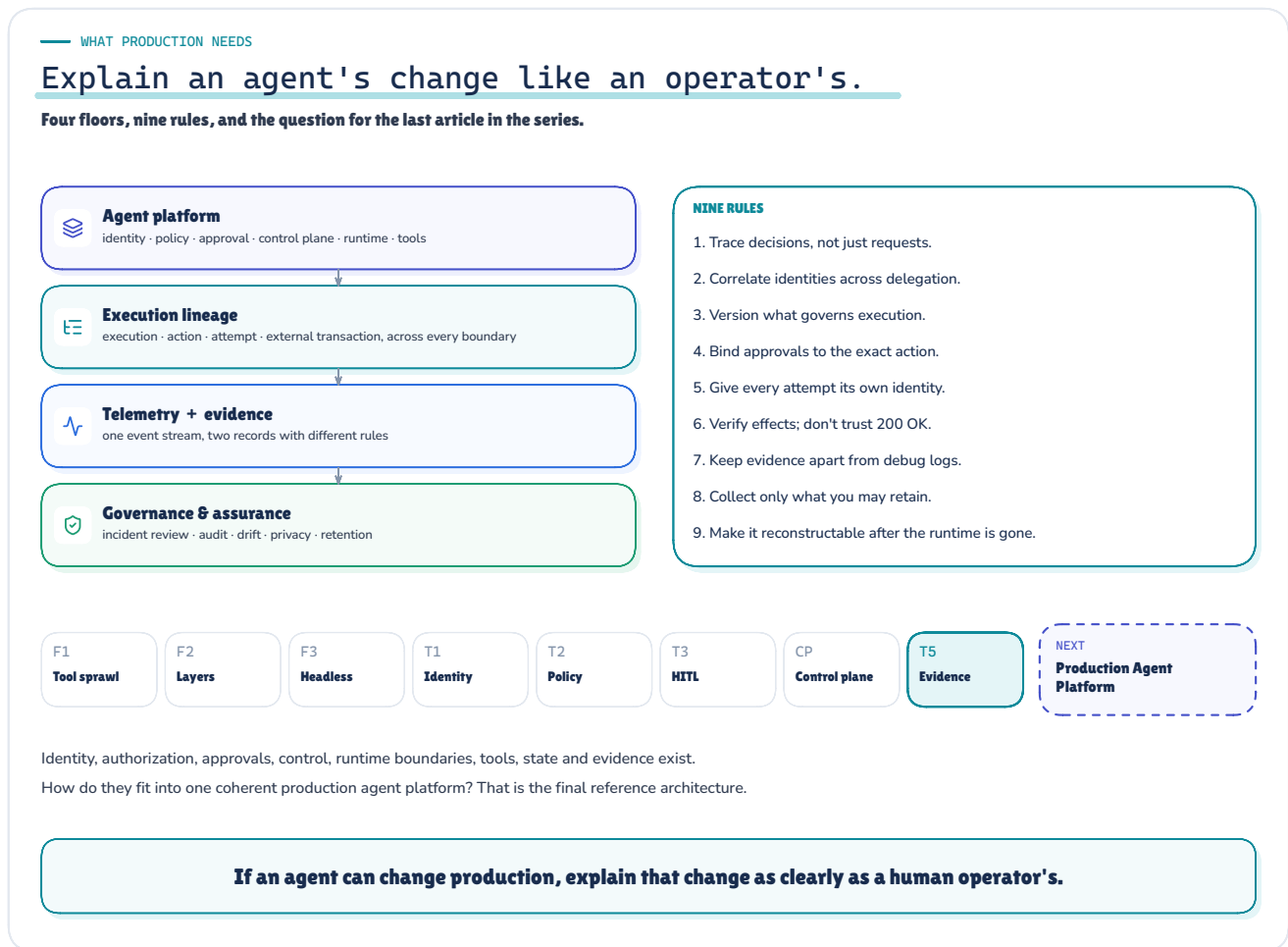


Figure 18. Four floors, nine rules, and the question for the last article in the series.

Architecture + series map: principles from the article; no measured values

1. **Trace decisions, not just requests.** Record the intent, the proposal, the policy decision and the approval as first-class events, not only the calls between services.
2. **Correlate identities across delegation boundaries.** The invoker, the principal it acts for, the agent, the workload and the tool identity belong in the same record as the action.
3. **Version the policies and configurations that govern execution.** Record what ran, with digests, at the moment it ran. The control plane's published state is not evidence.
4. **Bind approvals to the exact action they authorize.** An approval is evidence only if it names what it covers and can be checked against what executed.
5. **Give retries and attempts explicit identities.** An action has one identity; every try at it has another; the external system's transaction has a third, and all are recorded on both sides.
6. **Verify side effects instead of trusting tool responses.** Read the world back, from the system of record, and record what you saw and when.

7. **Treat audit evidence differently from debugging logs.** Different schema, retention, access, durability and integrity. Same events at the source.
8. **Collect only the evidence you are allowed to retain.** Schema-bound, digests over content, secrets refused, retention per event type.
9. **Make the production story reconstructable after the runtime is gone.** If the only way to know what happened is to ask the process that did it, you don't know what happened.

## 29. Next: the Production Agent Platform

This series has now built most of the platform one production problem at a time: a capability control plane for tools (F1), the layers (F2), headless invocation (F3), identity and delegation (T1), authorization (T2), durable human approval (T3), a control plane that coordinates them (the previous article), and now the evidence that lets anybody explain afterwards what all of it did.

We now have identity, authorization, approvals, control, runtime boundaries, tools, state and evidence. The final question is how all of those pieces fit into one coherent production agent platform: which boundaries are hard, which are conventions, what is deployed together, and what the whole thing looks like as a reference architecture. That's the last article: **Production Agent Platform — Final Reference Architecture**.

If an autonomous agent can change production, you should be able to explain that change as clearly as you would explain a human operator's action.

## References

Every source was fetched on 2026-09-30. `research/sources.md` has the verbatim passage each is cited for, and what each does *not* support.

[1] Semantic conventions for generative client AI spans (Inference span) — OpenTelemetry, semantic-conventions v1.41.1 (2026-05-11). [github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/operation-name.md](https://github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/operation-name.md). The inference/LLM-call span name is `{gen_ai.operation.name}` `{gen_ai.request.model}` (e.g. `chat llama3.2`), span kind CLIENT. The same wording appears in the new repo at commit bcc7f9c.

[2] Semantic conventions for generative client AI spans (Execute tool span) — OpenTelemetry, semantic-conventions v1.41.1 (2026-05-11). [github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/tool-execution.md](https://github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/tool-execution.md). The tool execution span is `execute_tool {gen_ai.tool.name}`, kind INTERNAL, with `gen_ai.operation.name = execute_tool` and `gen_ai.tool.name` Required.

`gen_ai.tool.call.id` is Recommended "if available". `gen_ai.tool.call.arguments` and `gen_ai.tool.call.result` are Opt-In and carry the warning "This attribute may contain sensitive information."

[3] Semantic conventions for generative client AI spans (Capturing instructions, inputs, and outputs) — OpenTelemetry, semantic-conventions v1.41.1 (2026-05-11). [github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md](https://github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md). Prompt/completion content is sensitive and off by default. `gen_ai.input.messages`, `gen_ai.output.messages` and `gen_ai.system_instructions` are Opt-In. The spec lists three patterns (default no content; content on span attributes; store content externally and record references), and recommends external storage for production. The same text is present in the new repo at bcc7f9c.

[4] Semantic Conventions for GenAI agent and framework spans (Invoke agent span) — OpenTelemetry, semantic-conventions v1.41.1 (2026-05-11). [github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md](https://github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md). The agent invocation span is `invoke_agent {gen_ai.agent.name}`, or just `invoke_agent` if no name. Kind is CLIENT for a remote agent and INTERNAL for an in-process agent. Attributes include `gen_ai.agent.id`, `gen_ai.agent.name`, `gen_ai.agent.version`, `gen_ai.agent.description` and `gen_ai.conversation.id`.

[5] Semantic Conventions for GenAI agent and framework spans (Create agent span) — OpenTelemetry, semantic-conventions v1.41.1 (2026-05-11). [github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md](https://github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md). A `create_agent` span exists for agent creation, with kind CLIENT.

[6] Gen AI attribute registry ( `gen_ai.system` ) — OpenTelemetry, semantic-conventions v1.41.1 (2026-05-11). [github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md](https://github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md). `gen_ai.system` is deprecated and replaced by `gen_ai.provider.name`, which is Required on inference and agent spans. Well-known values apply where one fits; otherwise "a custom value MAY be used", e.g. for Ollama, which has no listed value.

[7] Gen AI attribute registry ( `gen_ai.agent.version` ) — OpenTelemetry, semantic-conventions v1.41.1 (2026-05-11). [github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md](https://github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md). `gen_ai.agent.version` exists (string, examples `1.0.0` and `2025-05-01` ). It is Conditionally Required "when available" on `invoke_agent` spans in v1.41.1. The same attribute is present in the new-repo registry at bcc7f9c.

[8] Semantic conventions for generative client AI spans (transition note) — OpenTelemetry, semantic-conventions v1.41.1 (2026-05-11). [github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md](https://github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/gen-ai/README.md). Instrumentations that emit v1.36.0-or-prior GenAI conventions keep them by default. The newer conventions are opted into with `OTEL_SEMCONV_STABILITY_OPT_IN=gen_ai_latest_experimental`.

**[9]** Semantic conventions for generative AI systems (README) — OpenTelemetry, semantic-conventions v1.41.1 (2026-05-11). [github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/ge...](https://github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/ge...) The whole GenAI convention set (spans, agent spans, metrics, events) is at Development stability, not Stable. The new repo README at bcc7f9c also says "Development".

**[10]** OpenTelemetry GenAI Semantic Conventions repository (moved notice and current main) — OpenTelemetry, semantic-conventions-genai commit bcc7f9c (2026-09-29), no tagged release. [github.com/open-telemetry/semantic-conventions-genai/blob/bcc7f9c28...](https://github.com/open-telemetry/semantic-conventions-genai/blob/bcc7f9c28...) GenAI conventions now live in a separate repo. The [opentelemetry.io](https://opentelemetry.io) page <https://opentelemetry.io/docs/specs/semconv/gen-ai/> and core semconv v1.42.0+ show only a "Moved" stub. The unreleased main adds `invoke_workflow`, `plan`, `memory` spans and `gen_ai.request.reasoning.level`. Inference/tool/agent span names are unchanged from v1.41.1.

**[11]** Semantic conventions for generative AI metrics — OpenTelemetry, semantic-conventions v1.41.1 (2026-05-11). [github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/ge...](https://github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/ge...) Metric names `gen_ai.client.token.usage` (Histogram, `{token}`, requires `gen_ai.token.type = input / output`) and `gen_ai.client.operation.duration`.

**[12]** GenAI output messages JSON schema (ReasoningPart) — OpenTelemetry, semantic-conventions v1.41.1 (2026-05-11). [github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/ge...](https://github.com/open-telemetry/semantic-conventions/blob/v1.41.1/docs/ge...) When content capture is opted into, OTel has a message part type `reasoning` for reasoning/thinking content. `gen_ai.usage.reasoning.output_tokens` counts reasoning tokens. So the convention allows capturing reasoning content but does not require it.

**[13]** Trace Context (traceparent header) — W3C, Recommendation 23 November 2021. [www.w3.org/TR/trace-context/](https://www.w3.org/TR/trace-context/). The `traceparent` format is `version-trace-id-parent-id-trace-flags`, e.g. `00-0af7651916cd43dd8448eb211c80319c-b7ad6b7169203331-01`. `trace-id` is 16 bytes (32 lowercase hex) and `parent-id` is 8 bytes (16 hex). All zeros are invalid.

**[14]** Propagators API — OpenTelemetry Specification v1.61.0 (2026-09-14), Stable. [github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...](https://github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...) Context crosses process boundaries (HTTP, queues, subprocess env) via Propagator inject/extract. W3C TraceContext is the standard text-map propagator.

**[15]** Overview: Links between spans — OpenTelemetry Specification v1.61.0 (2026-09-14). [github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...](https://github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...) Span links, not parent/child, are the right tool for resumed/async work. Examples: an approval that resumes later in a new trace, or a reconciler that verifies a side effect. The Trace API requires the ability to add links at creation (preferred) or after.

**[16]** Logs Data Model (Trace Context Fields) — OpenTelemetry Specification v1.61.0 (2026-09-14), Stable. [github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...](https://github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...) Log records have optional `TraceId`, `SpanId` and `TraceFlags` fields for log-trace correlation. If `SpanId` is present, `TraceId` SHOULD be too.

**[17]** Tracing SDK: Sampling decision — OpenTelemetry Specification v1.61.0 (2026-09-14). [github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...](https://github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...) An unsampled (DROP) span is not recorded at all, so sampled-out traces cannot serve as audit evidence. Audit records must not depend on trace sampling.

**[18]** Tracing SDK: Built-in samplers (default) — OpenTelemetry Specification v1.61.0 (2026-09-14). [github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...](https://github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...) The default sampler is `ParentBased` with an `AlwaysOn` root. `ParentBased` respects the parent's sampled flag, so a child service follows the upstream head decision.

**[19]** Tracing SDK: `TraceIdRatioBased` (deprecation) — OpenTelemetry Specification v1.61.0 (2026-09-14). [github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...](https://github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...) `TraceIdRatioBased` (head, probabilistic) is deprecated in favor of the composable `ProbabilitySampler`. SDKs must keep it unchanged until at least 1 January 2027.

**[20]** Sampling (concepts) — OpenTelemetry documentation. [opentelemetry.io/docs/concepts/sampling/](https://opentelemetry.io/docs/concepts/sampling/). Head sampling decides early without seeing the whole trace. Tail sampling decides after seeing all or most spans, and needs stateful components (e.g. the Collector). Head vs. tail trade-offs.

**[21]** Metrics SDK: Cardinality limits — OpenTelemetry Specification v1.61.0 (2026-09-14), Stable. [github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...](https://github.com/open-telemetry/opentelemetry-specification/blob/v1.61.0/...) The SDK caps unique attribute combinations per metric (default 2000). Excess is folded into an overflow series with `otel.metric.overflow=true`. So per-run or per-call IDs belong on spans/logs, not metric attributes.

**[22]** The Idempotency-Key HTTP Header Field (draft-ietf-httpapi-idempotency-key-header-07) — IETF httpapi WG, Internet-Draft 15 October 2025. Datatracker (2026-09-30) shows "Expired & archived", WG Document, Intended RFC status None. [datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/](https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/). The idempotency-key pattern: a client-generated unique key lets the server recognize retries of the same request. A UUID is recommended. The resource defines key expiry.

**[23]** The Idempotency-Key HTTP Header Field, draft -07, sections 2.7 (Error Handling) — IETF httpapi WG, 15 October 2025 (expired). [www.ietf.org/archive/id/draft-ietf-httpapi-idempotency-key-header-0...](https://www.ietf.org/archive/id/draft-ietf-httpapi-idempotency-key-header-0...) A concurrent retry while the original is in flight gets 409 Conflict. Reusing a key with a different payload gets 422, detected via an optional request "fingerprint" (payload checksum).

[24] The Idempotency-Key HTTP Header Field, draft -07 (payload mismatch) — IETF httpapi WG, 15 October 2025 (expired). [www.ietf.org/archive/id/draft-ietf-httpapi-idempotency-key-header-0....](https://www.ietf.org/archive/id/draft-ietf-httpapi-idempotency-key-header-0000.html) A key reused with a different payload is an error, not a replay. This supports binding the idempotency key to a hash of the intended action.

[25] RFC 9110 HTTP Semantics, Section 9.2.2 Idempotent Methods — IETF, June 2022 (Internet Standard, STD 97). [www.rfc-editor.org/rfc/rfc9110.html#section-9.2.2](https://www.rfc-editor.org/rfc/rfc9110.html#section-9.2.2). The definition of idempotency (repeated identical requests have the same intended effect). PUT, DELETE and safe methods are idempotent. Non-idempotent requests must not be auto-retried without some means of knowing it is safe.

[26] Deployments: Rolling Back a Deployment — Kubernetes documentation (kubernetes/website main, retrieved 2026-09-30). [kubernetes.io/docs/concepts/workloads/controllers/deployment/#rolli....](https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#rollback) `kubectl rollout undo` does not restore the old revision number. A rollback produces a new revision. The docs' worked example rolls back "to revision 2" and the Deployment then shows `deployment.kubernetes.io/revision=4`, with a `DeploymentRollback` event. Revisions are created only when `.spec.template` changes, so scaling does not create one.

[27] Deployments: Revision History Limit — Kubernetes documentation (retrieved 2026-09-30). [kubernetes.io/docs/concepts/workloads/controllers/deployment/#revis....](https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#revision-history) Revision history lives in old ReplicaSets (default 10 kept). Once pruned, that revision cannot be rolled back to, so Kubernetes rollout history is not a durable audit log.

[28] Auditing — Kubernetes documentation (kubernetes/website main, retrieved 2026-09-30). [kubernetes.io/docs/tasks/debug/debug-cluster/audit/](https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/). kube-apiserver audit events and stages (`RequestReceived`, `ResponseStarted`, `ResponseComplete`, `Panic`). Audit levels (`None`, `Metadata`, `Request`, `RequestResponse`). Answers "who initiated it?" for API-server actions.

[29] Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, MEASURE 2.4 — NIST, January 2023. [nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf](https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf). Production monitoring of AI system behavior is an explicit RMF outcome.

[30] AI RMF 1.0, NIST AI 100-1, MANAGE 4.1 — NIST, January 2023. [nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf](https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf). Post-deployment monitoring should include override, incident response, recovery and change management. This maps to approval evidence and rollback records.

[31] Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1 (Appendix A, Incident Disclosure) — NIST, July 2024. [nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf](https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf). Logging and recording GAI incidents, plus change-management records, version history and metadata, supports incident response.

**[32]** NIST AI 600-1, action MG-2.2-007 — NIST, July 2024. [nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf](https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf). Lineage/provenance tracking of AI-generated data is a suggested action.

**[33]** Guide to Computer Security Log Management, NIST SP 800-92 (section 4, log file integrity) — NIST, September 2006. [nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-92.pdf](https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-92.pdf). Hashing (message digests) to detect modification of archived logs. The original digests must themselves be protected (read-only media, etc.), which is the anchoring problem.

**[34]** Regulation (EU) 2024/1689 (Artificial Intelligence Act), Article 12(1) Record-keeping — Official Journal of the EU, L series, 12.7.2024. [eur-lex.europa.eu/eli/reg/2024/1689/oj](https://eur-lex.europa.eu/eli/reg/2024/1689/oj). Providers must design high-risk AI systems to automatically record events (logs) over their lifetime.

**[35]** Regulation (EU) 2024/1689, Article 12(2) — Official Journal of the EU, 12.7.2024. [eur-lex.europa.eu/eli/reg/2024/1689/oj](https://eur-lex.europa.eu/eli/reg/2024/1689/oj). The purpose of the logs is traceability: identifying risk situations or substantial modifications, post-market monitoring (Art. 72), and deployer monitoring (Art. 26(5)).

**[36]** Regulation (EU) 2024/1689, Article 19(1) Automatically generated logs — Official Journal of the EU, 12.7.2024. [eur-lex.europa.eu/eli/reg/2024/1689/oj](https://eur-lex.europa.eu/eli/reg/2024/1689/oj). Providers keep the automatically generated logs under their control for at least six months, unless other Union/national law (notably data protection law) provides otherwise.

**[37]** Regulation (EU) 2024/1689, Article 26(6) Obligations of deployers — Official Journal of the EU, 12.7.2024. [eur-lex.europa.eu/eli/reg/2024/1689/oj](https://eur-lex.europa.eu/eli/reg/2024/1689/oj). Deployers also keep logs under their control for at least six months, subject to data-protection law.

**[38]** Regulation (EU) 2026/1744 (Digital Omnibus on AI), amending Article 113 of Regulation (EU) 2024/1689 — Official Journal of the EU, L series, 24.7.2026 (in force 27.7.2026). [eur-lex.europa.eu/eli/reg/2026/1744/oj](https://eur-lex.europa.eu/eli/reg/2026/1744/oj). Chapter III Sections 1-3 (which include Arts. 12, 19 and 26) now apply from 2 December 2027 for Annex III high-risk systems and 2 August 2028 for Annex I systems, not 2 August 2026. The omnibus did not change the wording of Arts. 12, 19 or 26 (checked against the omnibus's list of amended articles).

**[39]** Regulation (EU) 2016/679 (GDPR), Article 5(1)(c) data minimisation — Official Journal of the EU, L 119, 4.5.2016. [eur-lex.europa.eu/eli/reg/2016/679/oj](https://eur-lex.europa.eu/eli/reg/2016/679/oj). Telemetry that contains personal data (prompts, tool arguments, user IDs) should be limited to what is necessary. This supports default-off content capture and hashing/redaction.

**[40]** Regulation (EU) 2016/679 (GDPR), Article 5(1)(e) storage limitation — Official Journal of the EU, L 119, 4.5.2016. [eur-lex.europa.eu/eli/reg/2016/679/oj](https://eur-lex.europa.eu/eli/reg/2016/679/oj). Identifiable telemetry needs bounded retention. This creates tension with audit retention (AI Act Art. 19 defers to data-protection law), which is resolved by separating content from evidence and pseudonymizing.

[41] RFC 9162 Certificate Transparency Version 2.0 — IETF, December 2021 (Experimental; obsoletes RFC 6962). [www.rfc-editor.org/rfc/rfc9162.html](http://www.rfc-editor.org/rfc/rfc9162.html). Merkle-tree append-only logs with inclusion and consistency proofs. The limitation: a log that shows different views to different clients (split view) defeats auditing unless there is external gossip/witnessing, so the log must otherwise be treated as trusted.

[42] Efficient Data Structures for Tamper-Evident Logging (Crosby & Wallach), Section 1 — USENIX Security Symposium 2009. [www.usenix.org/legacy/event/sec09/tech/full\\_papers/crosby.pdf](http://www.usenix.org/legacy/event/sec09/tech/full_papers/crosby.pdf). Hash-chain/signed-commitment logs detect tampering within a snapshot, but an untrusted logger can present inconsistent histories (fork/rollback). Hash-chain audits cost linear time, and history trees are logarithmic.

[43] Efficient Data Structures for Tamper-Evident Logging (Crosby & Wallach), Section 2.2 — USENIX Security 2009. [www.usenix.org/legacy/event/sec09/tech/full\\_papers/crosby.pdf](http://www.usenix.org/legacy/event/sec09/tech/full_papers/crosby.pdf). Tamper evidence needs commitments to be distributed to at least one honest external auditor (gossip/witness/publication). This is the "external anchor" requirement for the POC's hash chain.

[44] LLM06:2025 Excessive Agency — OWASP Top 10 for LLM Applications 2025 (OWASP GenAI Security Project). [genai.owasp.org/llmrisk/llm062025-excessive-agency/](https://genai.owasp.org/llmrisk/llm062025-excessive-agency/). Human approval for high-impact actions. Complete mediation, meaning authorization is enforced in downstream systems and not by the LLM. Logging and monitoring of extension/downstream activity. Root causes are excessive functionality, permissions and autonomy.

[45] LLM06:2025 Excessive Agency, mitigations (complete mediation and monitoring) — OWASP Top 10 for LLM Applications 2025. [genai.owasp.org/llmrisk/llm062025-excessive-agency/](https://genai.owasp.org/llmrisk/llm062025-excessive-agency/). Policy enforcement outside the model, plus logging/monitoring of downstream actions.

[46] LLM02:2025 Sensitive Information Disclosure — OWASP Top 10 for LLM Applications 2025. [genai.owasp.org/llmrisk/llm022025-sensitive-information-disclosure/](https://genai.owasp.org/llmrisk/llm022025-sensitive-information-disclosure/). LLM apps can leak PII, credentials and confidential data through outputs. Mitigations include sanitization, tokenization/redaction, and clear retention/usage/deletion policies. This supports keeping raw content out of general-purpose telemetry.

[47] Thinking (Summarized thinking; Thinking encryption) — Anthropic, Claude Platform docs (retrieved 2026-09-30). [platform.claude.com/docs/en/build-with-claude/thinking](https://platform.claude.com/docs/en/build-with-claude/thinking). The Claude API never returns raw chain of thought. Thinking blocks are summaries, or empty when `display: "omitted"`, which is the default on newer models. Full thinking is returned only encrypted in the `signature` field. Logging "the model's reasoning" from the Claude API therefore captures, at most, a summary.

[48] Reasoning models guide — OpenAI API docs (retrieved 2026-09-30). [developers.openai.com/api/docs/guides/reasoning](https://developers.openai.com/api/docs/guides/reasoning). OpenAI does not expose raw reasoning tokens.

Only summaries are available, and reasoning items can be carried as opaque `encrypted_content`. Reasoning tokens are billed as output tokens.

**[49]** Generate a chat message ( `POST /api/chat` ) response schema — Ollama documentation (retrieved 2026-09-30). [docs.ollama.com/api/chat](https://docs.ollama.com/api/chat). `/api/chat` final responses include `total_duration`, `load_duration`, `prompt_eval_count`, `prompt_eval_cached_count`, `prompt_eval_duration`, `eval_count`, `eval_duration`, `done` and `done_reason`. These map to `gen_ai.usage.input_tokens` (`prompt_eval_count`) and `gen_ai.usage.output_tokens` (`eval_count`). Durations are in nanoseconds.

**[50]** Usage — Ollama documentation (retrieved 2026-09-30). [docs.ollama.com/api/usage](https://docs.ollama.com/api/usage). The meaning of Ollama usage fields. Also, all timing values are in nanoseconds.

**[51]** List models ( `GET /api/tags` ) response schema — Ollama documentation (retrieved 2026-09-30). [docs.ollama.com/api/tags](https://docs.ollama.com/api/tags). `/api/tags` returns a `digest` per model. This can pin the exact local model artifact in lineage records, stronger than the `model:tag` name alone because tags are mutable.

AI Engineering

Observability

AI Governance

OpenTelemetry

Platform Engineering

PRODUCTION AI ENGINEERING · THE SERIES SO FAR

FOUNDATION

**F1 · MCP Tool Sprawl**

Your AI agent has 500 MCP tools. Now what?

FOUNDATION

**F2 · Layered Architecture**

Your agent works in a demo. Why does it break in production?

FOUNDATION

**F3 · Headless AI**

Your AI shouldn't live inside the UI

TRUST

**T1 · Agent Identity**

Who is acting, and on whose authority?

TRUST

**T2 · Authorization & Policy**

What is this agent actually allowed to do?

TRUST

**T3 · Human-in-the-Loop**

Authorized. Should it still act?

PREVIOUS

**AI Control Plane**

Who coordinates identity, policy, approval and budgets across every agent? (in progress)

CURRENT

**T5 · Observability & Governance**

Your agent changed production. Can you prove why?

NEXT

**Production Agent Platform**

The final reference architecture (planned)

[The full learning map \(Start Here\)](#)

# T5 · Production AI Engineering

Production AI Engineering · T5. Every measured number here is substituted at build time from observability\_governance\_poc/runs/2026-09-30-recorded/facts.json; the enterprise systems are simulated and the run is deterministic. Figures carry their provenance.

- [Medium edition](#)
- [Evidence Check](#)
- [POC README](#)
- [Run summary](#)
- [Lab Console](#)
- [Run report](#)
- [QA report](#)